

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 23 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM
49 //
50 // ---- 常见优化手段速查清单 ----
51 //
52 // 1. Coalesced Memory Access (合并访问)
53 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
54 //   - 否则产生多次事务 (最坏 32 次)
55 //
56 // 2. Tiling (分块)
57 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
58 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
59 //
60 // 3. Vectorized Memory Access (向量化)
61 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数
62 //   - float4 = 128-bit, 单条指令加载 16 bytes

```

```

63 //
64 // 4. Thread Tile (寄存器分块)
65 //   - 每个线程计算多个输出元素 (TM×TN)，提高计算密度
66 //   - 减少线程总数，降低同步开销
67 //
68 // 5. Bank Conflict Avoidance
69 //   - Shared memory 有 32 banks × 4 bytes
70 //   - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
71 //   - 解决方案：PAD (在每行末尾加 1 个元素打破对齐)
72 //
73 // 6. Pipeline / Double Buffering (流水线)
74 //   - cp.async 异步拷贝下一批数据时同时做当前批的计算
75 //   - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
76 //
77 // 7. Tensor Core (MMA / WGMMMA)
78 //   - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
79 //   - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
80 //
81 // 8. Warp Specialization (Hopper+)
82 //   - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
83 //   - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
84 //
85 // 9. TMA (Tensor Memory Accelerator, Hopper+)
86 //   - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
87 //   - 配合 cp.async.bulk 实现异步数据搬运
88 //
89 // ---- Roofline 分析公式 ----
90 //
91 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
92 //
93 // GEMM (M=N=K=4096):
94 //   FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
95 //   Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
96 //   AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
97 //   FP16 TC ≈ 295:1, FP32 ≈ 20:1)
98 //
99 // GEMV (M=4096, K=4096):
100 //   FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPs
101 //   Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 //   AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105 //
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 //   - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 //   - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //     注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 //   - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115 //
116 #include <algorithm>
117 #include <cuda_fp16.h>
118 #include <cuda_runtime.h>
119 #include <float.h>
120 #include <stdio.h>
121 #include <stdlib.h>
122 #include <math.h>
123 #include <cublas_v2.h>
124 #include <cuda.h>
125 #include <cuda/barrier>

```

```

126
127 #define WARP_SIZE 32
128 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
129 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
130 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
131
132 // =====
133 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
134 // =====
135
136 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
137 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
138 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
139 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
140 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
141 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
142 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
143 //
144 // 初始: 每个 lane 持有自己的值 v0..v7
145 // lane: 0 1 2 3 4 5 6 7
146 // val: v0 v1 v2 v3 v4 v5 v6 v7
147 //
148 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
149 //
150 //      ┌──────────┐
151 // 对: (0,4) (1,5) (2,6) (3,7)
152 //
153 // lane: 0 1 2 3 4 5 6 7
154 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
155 //
156 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
157 //
158 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
159 // 对: (0,2) (1,3) (4,6) (5,7)
160 //      ─── 前4个一组 ─── 后4个一组 ───
161 //
162 // lane: 0 1 2 3 4 5 6 7
163 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$  (每lane持有前一轮2个值的和再加本轮配对)
164 //      = v0+v2+v4+v6 ... (逐步归约)
165 //
166 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
167 //
168 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
169 // 对: (0,1) (2,3) (4,5) (6,7)
170 //
171 // lane: 0 1 2 3 4 5 6 7
172 // val:  $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
173 //
174 // mask=0: 循环终止, 归约完成。
175 //
176 // 关键性质:
177 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
178 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
179 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
180 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
181 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
182 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
183 template <const int kWarpSize = WARP_SIZE, typename T = float>
184 __device__ __forceinline__ T warp_reduce_sum(T val) {
185 #pragma unroll
186     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
187         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
188     }
189     return val;
190 }

```

```

188
189 // Warp Reduce Max — generic
190 template <const int kWarpSize = WARP_SIZE, typename T = float>
191 __device__ __forceinline__ T warp_reduce_max(T val) {
192 #pragma unroll
193     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
194         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
195     }
196     return val;
197 }
198
199 // =====
200 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
201 // =====
202
203 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
204 // 两级归约流程:
205 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
206 // 2. warp leader (lane=0) 写入 shared memory
207 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
208 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
209 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
210 template <const int NUM_THREADS = 256>
211 __device__ float block_reduce_sum(float val) {
212     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
213     int warp = threadIdx.x / WARP_SIZE;
214     int lane = threadIdx.x % WARP_SIZE;
215     static __shared__ float shared[NUM_WARPS];
216
217     float value = warp_reduce_sum<WARP_SIZE>(val);
218     if (lane == 0)
219         shared[warp] = value;
220     __syncthreads();
221     value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
222     value = warp_reduce_sum<NUM_WARPS>(value);
223     // 关键: broadcast 结果到所有线程, 后续用 result
224     // 做除法等操作时所有线程都能拿到
225     value = __shfl_sync(0xffffffff, value, 0, 32);
226     return value;
227 }
228
229 // Block Reduce Max — FP32 (增强版, 带 broadcast)
230 template <const int NUM_THREADS = 256>
231 __device__ float block_reduce_max(float val) {
232     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
233     int warp = threadIdx.x / WARP_SIZE;
234     int lane = threadIdx.x % WARP_SIZE;
235     static __shared__ float shared[NUM_WARPS];
236
237     float value = warp_reduce_max<WARP_SIZE>(val);
238     if (lane == 0)
239         shared[warp] = value;
240     __syncthreads();
241     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
242     value = warp_reduce_max<NUM_WARPS>(value);
243     value = __shfl_sync(0xffffffff, value, 0, 32);
244     return value;
245 }
246
247 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
248 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
249 // 跨 block 求和的常见模式, 适合 N 较大时使用;
250 // Grid: ((N + 255) / 256, 1, 1)

```

```

251 // Block: (256, 1, 1)
252 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
253 template <const int NUM_THREADS = 256>
254 __global__ void block_reduce_all(float *a, float *y, int N) {
255     int tid = threadIdx.x;
256     int idx = blockIdx.x * NUM_THREADS + tid;
257     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
258     __shared__ float reduce_smem[NUM_WARPS];
259
260     float sum = (idx < N) ? a[idx] : 0.0f;
261     int warp = tid / WARP_SIZE;
262     int lane = tid % WARP_SIZE;
263
264     sum = warp_reduce_sum<WARP_SIZE>(sum);
265     if (lane == 0)
266         reduce_smem[warp] = sum;
267     __syncthreads();
268
269     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
270     if (warp == 0)
271         sum = warp_reduce_sum<NUM_WARPS>(sum);
272     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
273         atomicAdd(y, sum);
274 }
275
276 // ---- Dot Product: y = sum(a[i] * b[i]) ----
277 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
278 // Grid: ((N + 255) / 256, 1, 1)
279 // Block: (256, 1, 1)
280 // source: LeetCUDA/kernels/dot-product/dot_product.cu
281 template <const int NUM_THREADS = 256>
282 __global__ void dot(float *a, float *b, float *y, int N) {
283     int tid = threadIdx.x;
284     int idx = blockIdx.x * NUM_THREADS + tid;
285     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
286     __shared__ float reduce_smem[NUM_WARPS];
287
288     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
289     int warp = tid / WARP_SIZE;
290     int lane = tid % WARP_SIZE;
291
292     prod = warp_reduce_sum<WARP_SIZE>(prod);
293     if (lane == 0)
294         reduce_smem[warp] = prod;
295     __syncthreads();
296
297     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
298     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
299         prod = warp_reduce_sum<NUM_WARPS>(prod);
300     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
301         atomicAdd(y, prod);
302 }
303
304 // Dot Product + float4
305 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
306 // Block: (64, 1, 1), 256/4=64
307 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
308 // source: LeetCUDA/kernels/dot-product/dot_product.cu
309 template <const int NUM_THREADS = 256 / 4>
310 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
311     int tid = threadIdx.x;
312     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
313     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;

```

```

314 __shared__ float reduce_smem[NUM_WARPS];
315
316 float4 reg_a = FLOAT4(a[idx]);
317 float4 reg_b = FLOAT4(b[idx]);
318 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
319                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
320                          : 0.0f;
321 int warp = tid / WARP_SIZE;
322 int lane = tid % WARP_SIZE;
323
324 prod = warp_reduce_sum<WARP_SIZE>(prod);
325 if (lane == 0)
326     reduce_smem[warp] = prod;
327 __syncthreads();
328
329 prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
330 if (warp == 0)
331     prod = warp_reduce_sum<NUM_WARPS>(prod);
332 if (tid == 0)
333     atomicAdd(y, prod);
334 }
335
336
337 // =====
338 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
339 // =====
340 // 面试要点:
341 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
342 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
343 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
344
345 // ---- ReLU: y = max(0, x) ----
346 // Grid: ((N + 255) / 256, 1, 1)
347 // Block: (256, 1, 1)
348 // source: LeetCUDA/kernels/relu/relu.cu
349 __global__ void relu(float *x, float *y, int N) {
350     int idx = blockIdx.x * blockDim.x + threadIdx.x;
351     if (idx < N)
352         y[idx] = fmaxf(0.0f, x[idx]);
353 }
354
355 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
356 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
357 // Grid: ((N + 255) / 256, 1, 1)
358 // Block: (64, 1, 1)
359 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
360 // source: LeetCUDA/kernels/relu/relu.cu
361 __global__ void relu_vec4(float *x, float *y, int N) {
362     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
363     if (idx < N) {
364         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
365         float4 reg_y;
366         reg_y.x = fmaxf(0.0f, reg_x.x);
367         reg_y.y = fmaxf(0.0f, reg_x.y);
368         reg_y.z = fmaxf(0.0f, reg_x.z);
369         reg_y.w = fmaxf(0.0f, reg_x.w);
370         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
371     }
372 }
373
374 // ---- Elementwise Add: c = a + b ----
375 // Grid: ((N + 255) / 256, 1, 1)
376 // Block: (256, 1, 1)

```

```

377 // source: LeetCUDA/kernels/elementwise/elementwise.cu
378 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
379     int idx = blockIdx.x * blockDim.x + threadIdx.x;
380     if (idx < N)
381         c[idx] = a[idx] + b[idx];
382 }
383
384 // Elementwise Add + float4 向量化
385 // Grid: ((N + 255) / 256, 1, 1)
386 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
387 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
388 // source: LeetCUDA/kernels/elementwise/elementwise.cu
389 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
390     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
391     if ((idx + 3) < N) {
392         float4 reg_a = FLOAT4(a[idx]);
393         float4 reg_b = FLOAT4(b[idx]);
394         float4 reg_c;
395         reg_c.x = reg_a.x + reg_b.x;
396         reg_c.y = reg_a.y + reg_b.y;
397         reg_c.z = reg_a.z + reg_b.z;
398         reg_c.w = reg_a.w + reg_b.w;
399         FLOAT4(c[idx]) = reg_c;
400     } else if (idx < N) {
401         for (int i = 0; (idx + i) < N; i++) {
402             c[idx + i] = a[idx + i] + b[idx + i];
403         }
404     }
405 }
406
407 // ---- Histogram: y[a[i]]++ ----
408 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
409 // Grid: ((N + 255) / 256, 1, 1)
410 // Block: (256, 1, 1)
411 // source: LeetCUDA/kernels/histogram/histogram.cu
412 __global__ void histogram(int *a, int *y, int N) {
413     int idx = blockIdx.x * blockDim.x + threadIdx.x;
414     if (idx < N)
415         atomicAdd(&(y[a[idx]]), 1);
416 }
417
418 // =====
419 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
420 // =====
421 // 面试要点:
422 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
423 //   online (1-pass, 增量更新)
424 //   - Online Softmax 是 FlashAttention 的数学基础
425 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
426 //   + variance)
427 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
428
429 // =====
430 // Phase 3a: Softmax — 三级递进 (面试核心考点)
431 // =====
432 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
433 // 回答线索: naive(溢出) → safe → online
434
435 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
436 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
437 // 问题: x 值很大时 exp(x) 溢出为 inf
438 template <const int NUM_THREADS = 256>
439 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL

```



```

440 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
441 // source: LeetCode/kernels/softmax/softmax.cu
442 __global__ void softmax_per_token(float *x, float *y, int N) {
443     const int tid = threadIdx.x;
444     const int idx = blockIdx.x * blockDim.x + tid;
445
446     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
447     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
448     if (idx < N)
449         y[idx] = exp_val / exp_sum;
450 }
451
452 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
453 // 面试重点: 为什么 Softmax 需要 Safe?
454 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
455 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
456 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
457 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
458 // Grid: (S, 1, 1)
459 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
460 // source: LeetCode/kernels/softmax/softmax.cu
461 template <const int NUM_THREADS = 256>
462 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
463     const int tid = threadIdx.x;
464     const int idx = blockIdx.x * blockDim.x + tid;
465
466     // Pass 1: block reduce max — 找最大值
467     float val = (idx < N) ? x[idx] : -FLT_MAX;
468     float max_val = block_reduce_max<NUM_THREADS>(val);
469
470     // Pass 2: exp(x - max) → block reduce sum
471     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
472     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
473
474     if (idx < N)
475         y[idx] = exp_val / exp_sum;
476 }
477
478 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
479 // 面试重点: Online Softmax 为什么重要?
480 //   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
481 //   - Online Softmax 用递推公式合并为 1 遍遍历
482 //   - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
483 //   - 正是 FlashAttention 中 "online rescaling":
484 //       
$$O_{\text{new}} = \text{diag}(\exp(m_{\text{old}} - m_{\text{new}})) * O_{\text{old}} + \exp(m_{\text{cur}} - m_{\text{new}}) * P@V$$

485 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
486
487 // ---- Online Softmax 辅助结构 ----
488 // MD struct: 存储 running max (m) 和 running denominator (d = Σexp(x - m))
489 //
490 // 两种使用场景 (公式不同, 但结果等价):
491 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
492 //       m_new = max(m_old, x_i)
493 //       d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
494 //   2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
495 //       m = max(m1, m2)
496 //       d = d1*exp(m1-m) + d2*exp(m2-m)
497 //       当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
498 //
499 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
500 struct __align__(8) MD {
501     float m; // running max
502     float d; // running denominator (sum of exp(x - max))

```

```

503 };
504
505 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
506 //
507 // 不同于单元元素增量更新公式 ( $m_{\text{new}} = \max(m_{\text{old}}, x_i)$ ;  $d_{\text{new}} = d_{\text{old}} \cdot \exp(m_{\text{old}} - m_{\text{new}}) + \exp(x_i - m_{\text{new}})$ ) ,
508 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
509 // (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
510 // (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
511 //
512 // 合并公式 (对称, 不依赖"新/旧"概念) :
513 //  $m = \max(m1, m2)$ 
514 //  $d = d1 \cdot \exp(m1 - m) + d2 \cdot \exp(m2 - m) \leftarrow m1 \geq m2$  时退化为  $d1 + d2 \cdot \exp(m2 - m1)$ 
515 //
516 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
517 template <const int kWarpSize = WARP_SIZE>
518 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
519 #pragma unroll
520     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
521         MD other;
522         other.m = __shfl_xor_sync(0xffffffff, value.m, mask, kWarpSize);
523         other.d = __shfl_xor_sync(0xffffffff, value.d, mask, kWarpSize);
524
525         bool value_bigger = (value.m > other.m);
526         MD bigger_m = value_bigger ? value : other;
527         MD smaller_m = value_bigger ? other : value;
528
529         // d_bigger 无需 rescale (其基准 max 已是全局 max) ,
530         // d_smaller 需 rescale:  $\exp(m_{\text{smaller}} - m_{\text{bigger}})$ 
531         value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
532         value.m = bigger_m.m;
533     }
534     return value;
535 }
536
537 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
538 // Grid: (S, 1, 1)
539 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
540 // source: LeetCUDA/kernels/softmax/softmax.cu
541 template <const int NUM_THREADS = 256>
542 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
543     int local_tid = threadIdx.x;
544     int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
545     const int WARP_NUM = NUM_THREADS / WARP_SIZE;
546     int warp_id = local_tid / WARP_SIZE;
547     int lane_id = local_tid % WARP_SIZE;
548
549     // 初始化: 每个线程持有一个 (max, denom) 对
550     MD val;
551     val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
552     val.d = global_tid < N ? 1.0f : 0.0f;
553
554     // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
555     __shared__ MD shared[WARP_NUM];
556     MD res = warp_reduce_md_op<WARP_SIZE>(val);
557
558     if (lane_id == 0)
559         shared[warp_id] = res;
560     __syncthreads();
561
562     // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
563     // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
564     if (local_tid < WARP_SIZE) {
565         MD block_res =

```

```

566     local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
567     block_res = warp_reduce_md_op<WARP_NUM>(block_res);
568     if (local_tid == 0) {
569         shared[0] = block_res;
570     }
571 }
572 __syncthreads();
573
574 // 用全局 max 和 denom 做最终 softmax
575 MD final_res = shared[0];
576 float d_total_inverse = __fdividef(1.0f, final_res.d);
577 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
578 if (global_tid < N) {
579     y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
580 }
581 }
582
583 // =====
584 // Phase 3b: RMS Normalization (1-pass reduce)
585 // =====
586 // 面试要点:
587 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
588 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
589 //   - Llama 系列使用 RMS Norm
590 //   - grid(N, K/K), block(K): 一行一个 block
591 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
592 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
593 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
594 template <const int NUM_THREADS = 128>
595 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
596     int tid = threadIdx.x;
597     int bid = blockIdx.x;
598     int idx = bid * blockDim.x + threadIdx.x;
599     const float epsilon = 1e-5f;
600
601     __shared__ float s_variance;
602     float value = (idx < N * K) ? x[idx] : 0.0f;
603     float variance = value * value;
604     variance = block_reduce_sum<NUM_THREADS>(variance);
605     if (tid == 0)
606         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
607     __syncthreads();
608     if (idx < N * K)
609         y[idx] = (value * s_variance) * g;
610 }
611
612 // RMS Norm + float4
613 // Grid: (N, 1, 1)
614 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
615 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
616 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
617 template <const int NUM_THREADS = 128 / 4>
618 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
619     int tid = threadIdx.x;
620     int bid = blockIdx.x;
621     int idx = (bid * blockDim.x + threadIdx.x) * 4;
622     const float epsilon = 1e-5f;
623
624     __shared__ float s_variance;
625     float4 reg_x = FLOAT4(x[idx]);
626     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
627                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
628                                     : 0.0f;

```

```

629 variance = block_reduce_sum<NUM_THREADS>(variance);
630 if (tid == 0)
631     s_variance = rsqrtf(variance / (float)K + epsilon);
632 __syncthreads();
633 float4 reg_y;
634 reg_y.x = reg_x.x * s_variance * g;
635 reg_y.y = reg_x.y * s_variance * g;
636 reg_y.z = reg_x.z * s_variance * g;
637 reg_y.w = reg_x.w * s_variance * g;
638 if (idx < N * K)
639     FLOAT4(y[idx]) = reg_y;
640 }
641
642 // =====
643 // Phase 3c: Layer Normalization (2-pass reduce)
644 // =====
645 // 面试要点:
646 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
647 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
648 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
649 // Grid: (N, 1, 1), 一行一个 block
650 // Block: (128, 1, 1), NUM_THREADS=K=128
651 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
652 template <const int NUM_THREADS = 128>
653 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
654     int tid = threadIdx.x;
655     int bid = blockIdx.x;
656     int idx = bid * blockDim.x + threadIdx.x;
657     const float epsilon = 1e-5f;
658
659     __shared__ float s_mean;
660     __shared__ float s_variance;
661     float value = (idx < N * K) ? x[idx] : 0.0f;
662
663     // Pass 1: compute mean
664     float sum = block_reduce_sum<NUM_THREADS>(value);
665     if (tid == 0)
666         s_mean = sum / (float)K;
667     __syncthreads(); // 必须等待 s_mean 对所有线程可见
668
669     // Pass 2: compute variance = (x - mean)2
670     float variance = (value - s_mean) * (value - s_mean);
671     variance = block_reduce_sum<NUM_THREADS>(variance);
672     if (tid == 0)
673         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
674     __syncthreads(); // 必须等待 s_variance 对所有线程可见
675
676     if (idx < N * K)
677         y[idx] = ((value - s_mean) * s_variance) * g + b;
678 }
679
680 // Layer Norm + float4
681 // Grid: (N, 1, 1)
682 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
683 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
684 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
685 template <const int NUM_THREADS = 128 / 4>
686 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
687                                 int K) {
688     int tid = threadIdx.x;
689     int bid = blockIdx.x;
690     int idx = (bid * blockDim.x + threadIdx.x) * 4;
691     const float epsilon = 1e-5f;

```

```

692
693 __shared__ float s_mean;
694 __shared__ float s_variance;
695 float4 reg_x = FLOAT4(x[idx]);
696 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
697
698 float sum = block_reduce_sum<NUM_THREADS>(value);
699 if (tid == 0)
700     s_mean = sum / (float)K;
701 __syncthreads();
702
703 float4 reg_x_hat;
704 reg_x_hat.x = reg_x.x - s_mean;
705 reg_x_hat.y = reg_x.y - s_mean;
706 reg_x_hat.z = reg_x.z - s_mean;
707 reg_x_hat.w = reg_x.w - s_mean;
708 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
709                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
710 variance = block_reduce_sum<NUM_THREADS>(variance);
711 if (tid == 0)
712     s_variance = rsqrtf(variance / (float)K + epsilon);
713 __syncthreads();
714
715 float4 reg_y;
716 reg_y.x = reg_x_hat.x * s_variance * g + b;
717 reg_y.y = reg_x_hat.y * s_variance * g + b;
718 reg_y.z = reg_x_hat.z * s_variance * g + b;
719 reg_y.w = reg_x_hat.w * s_variance * g + b;
720 if (idx < N * K)
721     FLOAT4(y[idx]) = reg_y;
722 }
723
724 // =====
725 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
726 // =====
727 // 面试要点:
728 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
729 //     [x1']   [cos(θ)  -sin(θ)] [x1]
730 //     [x2'] = [sin(θ)   cos(θ)] [x2]
731 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
732 //   - token_pos = idx / N: token 在序列中的位置
733 //   - token_idx = idx % N: token 内的维度对索引
734 //   - 输入 [seq_len, hidden_size], 输出同形状
735
736 // Grid: ((seq_len * N + 255) / 256, 1, 1)
737 // Block: (256, 1, 1)
738 // source: LeetCUDA/kernels/rope/rope.cu
739 __global__ void rope(float *x, float *out, int seq_len, int N) {
740     int idx = blockIdx.x * blockDim.x + threadIdx.x;
741     float x1 = x[idx * 2];
742     float x2 = x[idx * 2 + 1];
743     int token_pos = idx / N; // 序列位置
744     int token_idx = idx % N; // 维度对索引
745
746     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
747     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
748     float sin_v = sinf(token_pos * exp_v);
749     float cos_v = cosf(token_pos * exp_v);
750
751     // 2D 旋转
752     float out1 = x1 * cos_v - x2 * sin_v;
753     float out2 = x1 * sin_v + x2 * cos_v;
754     out[idx * 2] = out1;

```

```

755     out[idx * 2 + 1] = out2;
756 }
757
758 // =====
759 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
760 // =====
761 // 面试要点 (Bank Conflict 专题) :
762 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
763 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
764 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
765 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
766 //
767 // 转置的四步演进:
768 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
769 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
770 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
771 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
772 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
773
774 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写) ----
775 // 每个线程处理 1 个元素, block(16,16)
776 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
777 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
778 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
779 // Block: (16, 16, 1)
780 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
781 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
782     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
783     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
784     if (global_y < row && global_x < col) {
785         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
786         // 写入: y[global_y][global_x] = y[global_y * row +
787         // global_x], 列优先, 非合并
788         y[global_y * row + global_x] = x[global_x * col + global_y];
789     }
790 }
791
792 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write) ----
793
794 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
795 // Block: (16, 16, 1)
796 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
797 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
798 __global__ void mat_transpose_padded(
799     float *x, float *y, const int row, const int col) {
800     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
801     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
802     const int local_x = threadIdx.x;
803     const int local_y = threadIdx.y;
804
805     constexpr int WARP_SIZE_S = 16;
806     constexpr int PAD = 1;
807     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
808     __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
809
810     if (global_y * 4 < row && global_x < col) {
811         // Step 1: 从 global memory 读取 4 行, 写入 shared
812         // memory (行优先, 合并读取)
813         float4 x_val;
814         x_val.x = x[(global_y * 4) * col + global_x];
815         x_val.y = x[(global_y * 4 + 1) * col + global_x];
816         x_val.z = x[(global_y * 4 + 2) * col + global_x];
817         x_val.w = x[(global_y * 4 + 3) * col + global_x];

```

```

818     tile[local_y * 4][local_x] = x_val.x;
819     tile[local_y * 4 + 1][local_x] = x_val.y;
820     tile[local_y * 4 + 2][local_x] = x_val.z;
821     tile[local_y * 4 + 3][local_x] = x_val.w;
822     __syncthreads();
823
824     // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
825     float4 smem_val;
826     smem_val.x = tile[local_x * 4][local_y];
827     smem_val.y = tile[local_x * 4 + 1][local_y];
828     smem_val.z = tile[local_x * 4 + 2][local_y];
829     smem_val.w = tile[local_x * 4 + 3][local_y];
830
831     const int gid_x = blockIdx.x * blockDim.x;
832     const int gid_y = blockIdx.y * blockDim.y * 4;
833     const int out_y = gid_y + local_x * 4;
834     const int out_x = gid_x + local_y;
835
836     // 逐元素写回 (避免 reinterpret_cast<float4> 的对齐要求)
837     y[out_x * row + out_y + 0] = smem_val.x;
838     y[out_x * row + out_y + 1] = smem_val.y;
839     y[out_x * row + out_y + 2] = smem_val.z;
840     y[out_x * row + out_y + 3] = smem_val.w;
841 }
842 }
843
844 // =====
845 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
846 // =====
847 // 面试要点:
848 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
849 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
850 //   - 不同 K 值对应不同分块策略:
851 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
852 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
853 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
854 //
855 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
856
857 // ---- SGEMV K32: 基础 warp-per-row ----
858 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
859 // grid(M/4), 每个 warp 负责一行
860 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
861 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
862 // Block: (32, 4, 1), 每行 1 warp
863 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
864 // source: LeetCode/kernels/sgemv/sgemv.cu
865 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
866     int tx = threadIdx.x; // 0~31
867     int ty = threadIdx.y; // 0~3
868     int bx = blockIdx.x;
869     int lane = tx % WARP_SIZE; // 0~31
870     int m = bx * blockDim.y + ty; // 全局行号
871     if (m < M) {
872         float sum = 0.0f;
873         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
874         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
875         const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
876         #pragma unroll
877         for (int w = 0; w < NUM_ITERS; ++w) {
878             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
879             int k = w * WARP_SIZE + lane;
880             sum += a[m * K + k] * x[k];

```



```

881     }
882     sum = warp_reduce_sum<WARP_SIZE>(sum);
883     // 每个 warp 处理一行, lane 0 写回结果
884     if (lane == 0)
885         y[m] = sum;
886 }
887 }
888
889 // ---- SGEMV K128: float4 向量化 ----
890 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
891 // Grid: ((M + 3) / 4, 1, 1)
892 // Block: (32, 4, 1)
893 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
894 // source: LeetCUDA/kernels/sgemv/sgemv.cu
895 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
896     int tx = threadIdx.x; // 0~31
897     int ty = threadIdx.y; // 0~3
898     int bx = blockIdx.x;
899     int lane = tx % WARP_SIZE; // 0~31
900     int m = blockDim.y * bx + ty;
901
902     if (m < M) {
903         float sum = 0.0f;
904         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
905         const int NUM_ITERS = ((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
906 #pragma unroll
907         for (int w = 0; w < NUM_ITERS; ++w) {
908             int k = (w * WARP_SIZE + lane) * 4;
909             float4 reg_x = FLOAT4(x[k]);
910             float4 reg_a = FLOAT4(a[m * K + k]);
911             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
912                    reg_a.w * reg_x.w);
913         }
914         sum = warp_reduce_sum<WARP_SIZE>(sum);
915         if (lane == 0)
916             y[m] = sum;
917     }
918 }
919
920 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
921 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
922 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
923 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
924 // Block: (32, 4, 1)
925 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
926 // source: LeetCUDA/kernels/sgemv/sgemv.cu
927 template <const int ROW_PER_WARP = 2>
928 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
929     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
930     int tx = threadIdx.x;
931     int ty = threadIdx.y;
932     int bx = blockIdx.x;
933     int lane = tx % WARP_SIZE;
934     int k = lane % K_WARP_SIZE; // 0~15
935     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
936     if (m < M) {
937         float sum = A[m * K + k] * x[k];
938         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
939         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
940         // 注意: 判断条件是 k == 0, 不是 lane == 0!
941         if (k == 0)
942             y[m] = sum;
943     }

```



```

944 }
945
946 // =====
947 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
948 // =====
949 // 面试要点 (GEMM 优化五层金字塔):
950 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
951 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
952 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
953 //   load/store 指令数 Level 4 — Tensor Core (MMA
954 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
955 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
956 //
957 // 计算密度递进:
958 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
959 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
960 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
961
962 // =====
963 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
964 // =====
965
966 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
967 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
968 // C = A x B, C[M, N] = A[M, K] x B[K, N]
969 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
970 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
971 // Block: (32, 32, 1), 1024 线程
972 // source: LeetCUDA/kernels/sgemm/sgemm.cu
973 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
974     constexpr int BM = 32; // vec 版: 32x4 = 128
975     constexpr int BN = 32; // vec 版: 32x4 = 128
976     constexpr int BK = 32;
977     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
978
979     int bx = blockIdx.x;
980     int by = blockIdx.y;
981     int tx = threadIdx.x;
982     int tid = threadIdx.y * blockDim.x + tx;
983
984     // 线程到 smem 的映射: 32×32 线程, 每个线程加载 a 和 b 各 1 个元素
985     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
986     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col
987     0~31, 每个线程加载 4 个元素, 8x4 = 32
988     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
989     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每
990     个线程加载 4 个元素, 32x4 = 128
991     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续)
992     [128x128]
993     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
994
995     float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32
996     x4, 32x4]=[128x128]大小
997     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
998         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
999         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1000         // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
1001         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1002         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1003         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1004         // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1005         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1006         __syncthreads(); // 确保整个 smem tile 加载完毕

```

```

1003
1004 #pragma unroll
1005     for (int k = 0; k < BK; ++k) {
1006         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1007         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1008         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1009     }
1010     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1011 }
1012 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1013 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1014 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1015 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1016 }
1017
1018 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1019 // 在 Level 1 基础上引入两层优化:
1020 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1021 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1022 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1023 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1024 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1025 // 1024 x 16 = 16384 = 128 x 128 ✓
1026 //
1027 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1028 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1029 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1030 //
1031 // 加载映射 (每线程 4 个元素, float4):
1032 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1033 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1034 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1035 //
1036 // △ Bank Conflict 提示 (面试加分点):
1037 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1038 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1039 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1040 //
1041 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1042 // Block: (32, 32, 1), 1024 线程
1043 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1044 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1045 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1046     constexpr int BM = 128;
1047     constexpr int BN = 128;
1048     constexpr int BK = 32;
1049     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1050     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1051
1052     int bx = blockIdx.x;
1053     int by = blockIdx.y;
1054     int tx = threadIdx.x;
1055     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1056
1057     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1058     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1059     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1060     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1061     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1062     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1063
1064     int load_gmem_a_m = by * BM + load_smem_a_m;
1065     int load_gmem_b_n = bx * BN + load_smem_b_n;

```

```

1066 // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
1067 int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1068 int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1069
1070
1071 float sum[4][4] = {0.f};
1072 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1073     int load_gmem_a_k = bk * BK + load_smem_a_k;
1074     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1075     FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
1076     int load_gmem_b_k = bk * BK + load_smem_b_k;
1077     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1078     FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
1079     __syncthreads();
1080
1081 #pragma unroll
1082     for (int k = 0; k < BK; ++k) {
1083         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1084         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1085                             s_a[comp_smem_a_m_base + 1][k],
1086                             s_a[comp_smem_a_m_base + 2][k],
1087                             s_a[comp_smem_a_m_base + 3][k]};
1088         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1089                             s_b[k][comp_smem_b_n_base + 1],
1090                             s_b[k][comp_smem_b_n_base + 2],
1091                             s_b[k][comp_smem_b_n_base + 3]};
1092 #pragma unroll
1093         for (int i = 0; i < 4; ++i) {
1094             #pragma unroll
1095             for (int j = 0; j < 4; ++j) {
1096                 sum[i][j] += a_vals[i] * b_vals[j];
1097             }
1098         }
1099     }
1100     __syncthreads();
1101 }
1102
1103 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1104 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1105 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1106 #pragma unroll
1107 for (int i = 0; i < 4; ++i) {
1108     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1109     float4 reg_c;
1110     reg_c.x = sum[i][0];
1111     reg_c.y = sum[i][1];
1112     reg_c.z = sum[i][2];
1113     reg_c.w = sum[i][3];
1114     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1115 }
1116 }
1117
1118 // =====
1119 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1120 // =====
1121 // 面试要点:
1122 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1123 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1124 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1125 //   寄存器)
1126 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1127 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1128 //

```

```

1129 // ★ TN 布局详解 (面试高频考点) :
1130 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1131 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1132 // N = Normal (列优先, BLAS 原生格式)
1133 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1134 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1135 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1136 // BLAS=Normal) 即: C = op(A) × op(B) = AT × B? 不对! 在 row-major
1137 // 视角下: TN = A row-major [M×K], BT row-major [N×K] (等价于 B col-major [K×N]) 在 cuBLAS
1138 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1139 // T on A: BLAS 把 row-major 的 A 视为 AT, 传 T 表示"转置回去"
1140 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1141 //
1142 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1143 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1144 // N = col-major (列优先, BLAS native = normal)
1145 //
1146 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[M×N] = A[M×K] × B[K×N]
1147 // - A: row-major [M, K], B: row-major [K, N]
1148 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1149 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1150 //
1151 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 BT=[N×K] row-major 存储 (A 行优先, B 列优先)
1152 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1153 // - BT [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1154 // - 优势: BT 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1155 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1156 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1157 //
1158 // WGMMA (Warp Group MMA, Hopper+):
1159 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1160 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1161 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1162 // threads) 做计算
1163 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1164 //
1165 // =====
1166 // Phase 5b-1: MMA PTX 宏定义
1167 // =====
1168 //
1169 // ---- gmem → smem: cp.async ----
1170 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1171 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1172 #define CP_ASYNC_WAIT_GROUP(n) \
1173     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1174 //
1175 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1176 #define CP_ASYNC_CG(dst, src, bytes) \
1177     asm volatile( \
1178         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1179         "l"(src), "n"(bytes))
1180 //
1181 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1182 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1183 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1184 // aligned: 要求 128-bit 对齐
1185 // trans: 转置加载 (用于 col-major 的 B 矩阵)
1186 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1187     asm volatile( \
1188         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1189         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1190         : "r"(addr))
1191

```

```

1192 #define LDMATRIX_X2(R0, R1, addr) \
1193     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1194                 : "=r"(R0), "=r"(R1) \
1195                 : "r"(addr)) \
1196
1197 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1198 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1199 #define LDMATRIX_X2_T(R0, R1, addr) \
1200     asm volatile( \
1201         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1202         : "=r"(R0), "=r"(R1) \
1203         : "r"(addr)) \
1204
1205 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1206 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1207 // row.col: A 是 row-major, B 是 col-major
1208 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1209 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1210 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1211 #define HMMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1212     asm volatile( \
1213         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1214         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1215         : "=r"(RD0), "=r"(RD1) \
1216         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1217         "r"(RC1)) \
1218
1219 // ---- MMA 辅助函数 ----
1220 // div_ceil: 整数除法向上取整
1221 #define HOST_DEVICE_INLINE __device__ __host__ inline
1222 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1223     return (a % b != 0) ? (a / b + 1) : (a / b);
1224 }
1225
1226 // =====
1227 // Phase 5b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局
1228 // =====
1229 // 面试重点 — Tile Hierarchy:
1230 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1231 //   MMA Tile (warp):    2×4=8 个 MMA atom = [32×32] (一个 warp 的计算)
1232 //   Warp Tile:         4×4=16 warps = [128×128] (整个 block 的计算)
1233 //   线程映射:          2×4=8 warps, 每个 warp 4×1=4 个 warp tile
1234 //   实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
1235 //       → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1236 //
1237 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1238 //   - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1239 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major), 全局索引 B[n*K+k] 即原 B 元素 (k,n)
1240 //       (Δ 内维连续的是 K) shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1241 //   - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1242 //   - ldmatrix B: 用 x2 (非转置), 因为 B^T 在 smem 中为 row-major, ldmatrix 逐行加载
1243 //       B^T 的行即 B 的列, 天然匹配 MMA row.col 的 col-major B 输入, 无需 .trans
1244 //   - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1245 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1246 //   - grid.z = S 个 swizzle 分区
1247 //   - grid.x = 每个分区内部需要发射多少个 128×128 的 N tiles
1248 //   - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N 区域, 改善 L2 命中
1249 // Block: (256, 1, 1), 8 warps
1250 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1251 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1252         const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1253         const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1254         const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>

```

```

1255 __global__ void __launch_bounds__(256)
1256   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1257   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1258   const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1259   const int by = blockIdx.y;
1260   constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1261   constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1262   constexpr int BK = MMA_K; // 16
1263
1264   // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1265   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1266   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1267   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1268   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1269   extern __shared__ half smem[];
1270   half *s_a = smem;
1271   half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1272   constexpr int s_a_stage_offset = BM * BK; // 128*16
1273   constexpr int s_b_stage_offset =
1274       BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major 的 smem 布局
1275
1276   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1277   const int warp_id = tid / WARP_SIZE; // 0~7
1278   const int lane_id = tid % WARP_SIZE; // 0~31
1279   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1280   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1281
1282   // 线程到 global memory 的映射 (用于加载 A 和 B)
1283   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1284   int load_smem_a_m = tid / 2; // 0~127
1285   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1286   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1287   int load_smem_b_k =
1288       (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1289   int load_gmem_a_m = by * BM + load_smem_a_m;
1290   int load_gmem_b_n =
1291       bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏移
1292   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1293       return;
1294
1295   // 累加器: 每个 warp 计算 WARP_TILE_M×WARP_TILE_N=16 个 MMA tile
1296   uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1297
1298   // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1299   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1300   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1301
1302   // 预加载前 (K_STAGE-1) 个 stage
1303   // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1304   #pragma unroll
1305   for (int k = 0; k < (K_STAGE - 1); ++k) {
1306       int load_gmem_a_k = k * BK + load_smem_a_k;
1307       int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1308       int load_gmem_b_k = k * BK + load_smem_b_k;
1309       int load_gmem_b_addr =
1310           load_gmem_b_n * K + load_gmem_b_k; // B^T: [n][k] row-major (即 B[k][n] col-major) △
1311
1312       uint32_t load_smem_a_ptr =
1313           (smem_a_base_ptr +
1314            (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
1315             sizeof(half));
1316       CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1317

```



```

1318     uint32_t load_smem_b_ptr =
1319         (smem_b_base_ptr +
1320          (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
1321           sizeof(half));
1322     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1323
1324     CP_ASYNC.COMMIT_GROUP();
1325 }
1326
1327 const int NUM_K_TILES = div_ceil(K, BK);
1328 CP_ASYNC.WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1329 __syncthreads();
1330
1331 // 主循环: K 维分块迭代
1332 #pragma unroll
1333 for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1334     // Stage 选择: 轮转方式 (round-robin)
1335     // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1336     // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
1337     int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1338     int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1339
1340     // 异步加载下一批数据到 smem_sel_next
1341     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用
1342     // n*K+k (row-major, 内维连续的是 K)
1343     int load_gmem_a_k = k * BK + load_smem_a_k;
1344     int load_gmem_a_addr =
1345         load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1346     int load_gmem_b_k = k * BK + load_smem_b_k;
1347     int load_gmem_b_addr =
1348         load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k], 内维连续的是 K
1349
1350     uint32_t load_smem_a_ptr =
1351         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1352          load_smem_a_m * BK + load_smem_a_k) *
1353          sizeof(half));
1354     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1355
1356     uint32_t load_smem_b_ptr =
1357         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1358          load_smem_b_n * BK + load_smem_b_k) *
1359          sizeof(half));
1360     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1361     CP_ASYNC.COMMIT_GROUP();
1362
1363     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1364     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1365     // B 用 x2 (非转置), smem 中 B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1366     uint32_t RA[WARP_TILE_M][4];
1367     uint32_t RB[WARP_TILE_N][2];
1368
1369     // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1370 #pragma unroll
1371     for (int i = 0; i < WARP_TILE_M; ++i) {
1372         int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1373         int lane_smem_a_m =
1374             warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1375         int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1376         uint32_t lane_smem_a_ptr =
1377             (smem_a_base_ptr +
1378              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1379              sizeof(half));
1380         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);

```

```

1381 }
1382
1383 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1384 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1385 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1386 #pragma unroll
1387 for (int j = 0; j < WARP_TILE_N; ++j) {
1388     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1389     int lane_smem_b_n =
1390         warp_smem_b_n + lane_id % 8; // ldmatrix B 用 8 个 lane
1391     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1392     uint32_t lane_smem_b_ptr =
1393         (smem_b_base_ptr +
1394          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1395           sizeof(half));
1396     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1397 }
1398
1399 // MMA compute: 发射 WARP_TILE_M × WARP_TILE_N 条 MMA 指令
1400 #pragma unroll
1401 for (int i = 0; i < WARP_TILE_M; ++i) {
1402     #pragma unroll
1403     for (int j = 0; j < WARP_TILE_N; ++j) {
1404         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1405                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1406     }
1407 }
1408
1409 // 等待当前 stage 的异步加载完成, 然后 sync
1410 CP_ASYNC_WAIT_GROUP(K_STAGE - 2);
1411 __syncthreads();
1412 }
1413
1414 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1415 if ((K_STAGE - 2) > 0) {
1416     CP_ASYNC_WAIT_GROUP(0);
1417     __syncthreads();
1418 }
1419
1420 {
1421     #pragma unroll
1422     for (int k = 0; k < (K_STAGE - 1); k++) {
1423         uint32_t RA[WARP_TILE_M][4];
1424         uint32_t RB[WARP_TILE_N][2];
1425         int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);
1426
1427         #pragma unroll
1428         for (int i = 0; i < WARP_TILE_M; ++i) {
1429             int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1430             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1431             int lane_smem_a_k = (lane_id / 16) * 8;
1432             uint32_t lane_smem_a_ptr =
1433                 (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1434                                     lane_smem_a_m * BK + lane_smem_a_k) *
1435                  sizeof(half));
1436             LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1437         }
1438         #pragma unroll
1439         for (int j = 0; j < WARP_TILE_N; ++j) {
1440             int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1441             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1442             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1443             uint32_t lane_smem_b_ptr =

```



```

1444         (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1445                             lane_smem_b_n * BK + lane_smem_b_k) *
1446                             sizeof(half));
1447     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1448 }
1449 #pragma unroll
1450 for (int i = 0; i < WARP_TILE_M; ++i) {
1451     #pragma unroll
1452     for (int j = 0; j < WARP_TILE_N; ++j) {
1453         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1454                  RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1455     }
1456 }
1457 }
1458 }
1459
1460 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1461 {
1462     for (int i = 0; i < WARP_TILE_M; ++i) {
1463         uint32_t RC0[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1464         uint32_t RC1[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1465         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1466         // 对 m16n8k16 的 C fragment:
1467         //   - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1468         //   - lane+1 持有同一行里的 {c2,c3}
1469         //   - lane+2 持有同一行里的 {c4,c5}
1470         //   - lane+3 持有同一行里的 {c6,c7}
1471         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由 lane%4==0
1472         // 的那个线程做一次 128-bit store。
1473     #pragma unroll
1474     for (int j = 0; j < WARP_TILE_N; ++j) {
1475         RC0[j][0] = RC[i][j][0];
1476         RC1[j][0] = RC[i][j][1];
1477         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1478         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1479         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1480         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1481         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1482         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1483     }
1484     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1485     if (lane_id % 4 == 0) {
1486         int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1487         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1488     #pragma unroll
1489     for (int j = 0; j < WARP_TILE_N; ++j) {
1490         int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1491         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1492         int store_gmem_c_addr_0 =
1493             store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1494         int store_gmem_c_addr_1 =
1495             (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1496         // 128-bit store: 一次写入 8 个 half
1497         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1498             *reinterpret_cast<float4*>(&RC0[j][0]);
1499         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1500             *reinterpret_cast<float4*>(&RC1[j][0]);
1501     }
1502 }
1503 }
1504 }
1505 }
1506

```

```

1507 // =====
1508 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1509 // =====
1510 // 面试要点 (WGMMMA vs MMA 对比) :
1511 // - MMA: warp 级 (32 threads), 同步执行
1512 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
1513 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA 的 32
1514 // 倍)
1515 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1516 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier
1517 // 同步
1518 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1519
1520 // ---- WGMMMA 辅助函数 ----
1521 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1522 #define WGMMMA_COMMIT_GROUP() \
1523     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1524 #define WGMMMA_WAIT_GROUP(n) \
1525     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
1526
1527 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
1528 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
1529 //   encoded = (x & 0x3FFFF) >> 4
1530 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
1531 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
1532 // 可省略以扩大可表示范围。
1533 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
1534 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
1535
1536 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
1537 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
1538 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
1539 //
1540 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
1541 // -----
1542 // | 位域          | 范围      | 大小 | 含义
1543 // |-----|-----|-----|-----|
1544 // | start_address  | [0, 14)   | 14   | smem 基址编码
1545 // | (unused)       | [14, 16)  | 2    | -
1546 // | leading_byte_offset | [16, 30) | 14   | 次要维度的字节步长编码
1547 // | (unused)       | [30, 32)  | 2    | -
1548 // | stride_byte_offset | [32, 46) | 14   | 主要维度的字节步长编码
1549 // | (unused)       | [46, 49)  | 3    | -
1550 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
1551 // | (unused)       | [52, 62)  | 10   | -
1552 // | layout_type    | [62, 64)  | 2    | swizzle 模式
1553 // -----
1554 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
1555 //
1556 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
1557 // - 128B swizzle atom 在 128-bit 单位下为 8×8
1558 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
1559 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
1560 // - 这是 stride_byte_offset=1024 的来源
1561 //
1562 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
1563 //   此处填 16 仅为占位, 编码后为 1。
1564 //
1565 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
1566 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
1567 //
1568 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
1569 __device__ inline uint64_t make_smem_desc(half *ptr) {

```

```

1570 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
1571 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
1572 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
1573
1574 // 从全零开始: 所有位域初始化为 0。
1575 uint64_t desc = 0x0000000000000000;
1576
1577 // — bits [0, 14): start_address —
1578 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit),
1579 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
1580 // 可寻址范围:  $2^{14+4} = 2^{18} = 256K$  个 half = 512 KB 共享内存。
1581 desc |= SMEM_DESC_ENCODE(addr);
1582
1583 // — bits [16, 30): leading_byte_offset —
1584 // 原始值 16 (字节), 编码后为  $(16 \& 0x3FFFF) \gg 4 = 1$ 。
1585 // << 16 将编码值放到 bits [16, 30)。
1586 // K-Major + 128B swizzle 下该字段 unused (hardware assumes 1),
1587 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
1588 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
1589 // 对 INTERLEAVE: 同一 8x2 brick 内第一列到第二列的字节偏移。
1590 // 对 swizzle: unused, assumed to be 1。
1591 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
1592
1593 // — bits [32, 46): stride_byte_offset —
1594 // 原始值 1024 (字节), 编码后为  $(1024 \& 0x3FFFF) \gg 4 = 64$ 。
1595 // << 32 将编码值放到 bits [32, 46)。
1596 // 1024 的来源 (K-Major + 128B swizzle + half):
1597 // 一个 128B swizzle atom 覆盖  $64(K) \times 8(M)$  个 half。
1598 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越  $8 \times 64 \times 2 = 1024$  bytes。
1599 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组)。
1600 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
1601
1602 // — bits [62, 64): layout_type (swizzle mode) —
1603 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
1604 // 1 = 128B swizzle mode。
1605 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
1606 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
1607 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
1608 desc |= 1llu << 62;
1609
1610 return desc;
1611 }
1612
1613 // ---- WGMMA PTX 指令宏 ----
1614 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1615 // m64n128k16: M=64, N=128, K=16
1616 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1617 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
1618 // 每线程 64 half = 32 uint32
1619 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1620 // Scaled: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1621 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, Scaled, ScaleA, ScaleB, TransA, \
1622                                     TransB) \
1623 { \
1624     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1625     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1626     asm volatile( \
1627         "{\n" \
1628         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1629         "{%0,  %1,  %2,  %3,  %4,  %5,  %6,  %7,  " \
1630         " %8,  %9,  %10, %11, %12, %13, %14, %15, " \
1631         " %16, %17, %18, %19, %20, %21, %22, %23, " \
1632         " %24, %25, %26, %27, %28, %29, %30, %31}, " \

```

```

1633     " %32," \
1634     " %33," \
1635     " %34, %35, %36, %37, %38;\n" \
1636     "}\n" \
1637     : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1638     "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1639     "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1640     "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1641     "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1642     "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1643     "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1644     "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1645     : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
1646     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1647     "n"(int32_t(TransB)); \
1648 }
1649
1650 // ---- TMA Shared Memory Layout ----
1651 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1652 //
1653 // 每个 stage 包含两块 smem:
1654 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
1655 //   B tile: [BK, BN] row-major → BN×BK 个 half, 地址连续
1656 // 注意 B 的 smem 布局是 row-major [BK, BN], 物理上按 K-major 排列 (imm-trans-b=0),
1657 // 与 A 的 K-major 配合供 WGMMMA 读取。128B swizzle 将 [BK,BN] row-major 重新映射为
1658 // K-major 布局, 使得 K 维元素在 128B atom 内连续, 区别于 MMA TN 布局下的 B^T[N,K]。
1659 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
1660     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1661     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (K-major 布局, 供 WGMMMA imm-trans-b=0 直
        接读取)
1662 };
1663
1664 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1665 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
1666 //
1667 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
1668 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
1669 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
1670 //
1671 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
1672 //   将 A/B tile 从 HBM 搬到 shared memory。
1673 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
1674 //
1675 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
1676 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪
1677 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可被覆盖
1678 //
1679 // 本节重点理解:
1680 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
1681 //       即可搬运整个 2D tile, 无需所有线程参与。
1682 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
1683 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
1684 //       Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
1685 //
1686 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
1687 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
1688 //   K Tile:           BK=64, 每个 K tile 包含 BK/WGMMMA_K=4 个 WGMMMA atom
1689 //   M Tile:           BM=128, 每个 M tile 包含 BM/WGMMMA_M=2 个 WGMMMA atom
1690 //   N Tile:           BN=128, 单个 WGMMMA_N=128 即可覆盖, 无需在 N 方向分块
1691 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
1692 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
1693 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
1694 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)

```

```

1695 //
1696 // K_STAGE=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步，确保 HBM-SMEM
1697 // 的延迟被计算完全掩盖。
1698 //
1699 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1700 // - grid.z = S 个 swizzle 分区，将连续 block 打散到不同 N 区域改善 L2 命中
1701 // Block: (256, 1, 1), 2 warpgroups
1702 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1703 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1704          const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1705          const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1706          const bool BLOCK_SWIZZLE = false>
1707 __global__ void __launch_bounds__(NUM_THREADS)
1708 hgemm_wgmma_stages_tn(
1709     int M, int N, int K, half *C,
1710     const CUtensorMap *__restrict__ tensorMapA,
1711     const CUtensorMap *__restrict__ tensorMapB) {
1712
1713     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1714     // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1715     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1716     // 不展开宿主侧 create_tensor_map 细节。
1717
1718     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1719     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
1720     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1721     const int by = blockIdx.y;
1722     // num_consumers = (NUM_THREADS / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
1723     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
1724     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1725     // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
1726     // 当只有一个 consumer 时, 它负责全部 BM=128 行
1727     constexpr int B_WG_M = BM / num_consumers; // 128
1728
1729     // 边界检查: 确保当前 block 不超出 M/N 范围
1730     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1731         return;
1732
1733     // ---- Shared Memory 分配 ----
1734     // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
1735     // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
1736     extern __shared__ __align__(128) uint8_t smem_wgmma[];
1737     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1738         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem_wgmma);
1739     half *s_a = s.A;
1740     half *s_b = s.B;
1741
1742     // ---- cuda::barrier 初始化 ----
1743     // 每个 stage 有两个 barrier:
1744     // full[qidx]: Producer thread 0 发 full 信号, 128 个 Consumer 线程等 full
1745     // empty[qidx]: Consumer 发 empty 信号, Producer thread 0 等 empty
1746     // 每轮参与 arrive 的总人数 = 128 (consumer) + 1 (producer) = 129
1747 #pragma nv_diag_suppress static_var_with_dynamic_init
1748     __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1749     __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];
1750 #pragma nv_diag_default static_var_with_dynamic_init
1751
1752     // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
1753     const int num_blocks_k = K / BK;
1754     const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1755     const int tid = threadIdx.x % 128; // 0~127 within warpgroup
1756
1757     // 初始化 barriers (仅 thread 0 执行)

```

```

1758 if (threadIdx.x == 0) {
1759     for (int i = 0; i < K_STAGE; ++i) {
1760         // init 的第二个参数是 barrier 的 arrive count:
1761         // num_consumers * 128 + 1 = 1 * 128 + 1 = 129
1762         // 即每轮需要 128 个 consumer 线程 + 1 个 producer 线程都 arrive 后,
1763         // barrier 才翻转 phase 并唤醒等待线程。
1764         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1765         init(&empty[i], num_consumers * 128 + 1); // same
1766     }
1767     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
1768     // 对 async proxy (TMA/WGMMMA) 可见。
1769     cuda::device::experimental::fence_proxy_async_shared_cta();
1770 }
1771 __syncthreads();
1772
1773 // =====
1774 // Producer Warpgroup (WG0, threadIdx.x 0~127)
1775 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
1776 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
1777 // =====
1778 if (wg_idx == 0) {
1779     if (tid == 0) {
1780         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
1781         int qidx = 0;
1782         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1783              ++block_k_iter, ++qidx) {
1784             if (qidx == K_STAGE)
1785                 qidx = 0;
1786
1787             // Step P1: 等待 Consumer 释放此 stage (empty 信号)
1788             // empty[qidx].arrive() 表示 Producer 自己"已准备好等待",
1789             // 然后 wait() 阻塞直到 barrier phase 翻转 (即所有 Consumer 都已 arrive)。
1790             empty[qidx].wait(empty[qidx].arrive());
1791
1792             // Step P2: 提交 TMA 2D 拷贝指令
1793             // cp_async_bulk_tensor_2d_global_to_shared:
1794             // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
1795             // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
1796             // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
1797             // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
1798             //
1799             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
1800             // 无需线程逐元素搬运, 零寄存器开销。
1801             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1802             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1803                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1804                 full[qidx]);
1805
1806             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1807             cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
1808                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1809                 full[qidx]);
1810
1811             // Step P3: 通知 Consumer 此 stage 已准备好 (full 信号)
1812             // barrier_arrive_tx 除了 arrive 之外还额外告知硬件:
1813             // 本次 TMA 事务预期传输的字节数。
1814             // Consumer 的 full[qidx].wait() 会等待这 (BK*BN+BK*BM)*sizeof(half)
1815             // 字节全部写入 smem 后才返回。
1816             [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
1817                 full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
1818         }
1819     }
1820 }

```



```

1821 // =====
1822 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
1823 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
1824 // 所有 128 个线程 (4 warps) 全部参与。
1825 // =====
1826 else {
1827     // Step C0: Consumer 初始时"准备就绪"
1828     // 对所有 stage 的 empty barrier 执行 arrive, 表示初始时所有 stage
1829     // 都是"空的" (可被 Producer 写入)。
1830     // 如果没有这一步, Producer 的 empty[qidx].wait() 在第一轮会永远阻塞。
1831     for (int i = 0; i < K_STAGE; ++i) {
1832         [[maybe_unused]] auto token = empty[i].arrive();
1833     }
1834
1835     // 累加器寄存器声明
1836     // d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4]:
1837     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
1838     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
1839     //   - d[*][g][*]: N 方向第 g 组 16 列
1840     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
1841     // 每个线程总共 2 * 8 * 4 = 64 uint32 = 128 half
1842     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
1843     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1844
1845     int qidx = 0;
1846     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
1847     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1848         ++block_k_iter, ++qidx) {
1849         if (qidx == K_STAGE)
1850             qidx = 0;
1851
1852         // Step C1: 等待 Producer 的 full 信号
1853         // 表示 stage qidx 的 A/B tile 数据已通过 TMA 搬运到 smem。
1854         full[qidx].wait(full[qidx].arrive());
1855
1856         // Step C2: 发射 WGMMMA 指令序列
1857         //
1858         // WGMMMA 指令序列的固定模式:
1859         //   WGMMMA_FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP
1860         //
1861         // WGMMMA_FENCE: 确保 smem 写可见、accum 寄存器准备好。
1862         //   是一条内存 fence 指令, 建立 async proxy 与 generic proxy 之间的
1863         //   可见性顺序。必须在发射 WGMMMA 之前执行。
1864         //
1865         // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
1866         //   执行 D[64*128] += A[64*16] * B[16*128]
1867         //   其中 A 的 smem 指针由 descA 描述 (K-major), B 由 descB 描述 (K-major)
1868         //   imm-trans-a=0/imm-trans-b=0 表示 A/B 都是 K-major (非转置)。
1869         //   ScaleD=1: 累加模式 (不清零), 用于 K 维累加。
1870         //   ScaleA=ScaleB=1: 不翻转符号。
1871         WGMMMA_FENCE();
1872
1873         // M 维迭代: BM/WGMMMA_M = 128/64 = 2 个 WGMMMA atom
1874         // 每个 atom 处理 64 行 M 方向数据。
1875 #pragma unroll
1876         for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1877             // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
1878             // s_a 布局: [K_STAGE][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
1879             half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1880
1881             // K 维迭代: BK/WGMMMA_K = 64/16 = 4 次 WGMMMA (累加)
1882             // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
1883 #pragma unroll

```



```

1884     for (int k_it = 0; k_it < BK / WGMMA_K; ++k_it) {
1885         // 第 k_it 次 WGMMA:
1886         //   A: wgmma_sA + k_it * WGMMA_K (A 的 K 维起始位置)
1887         //   B: s_b + qidx * BK * BN + k_it * WGMMA_K (B 的 K 维起始位置)
1888         //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
1889         //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
1890         //   ScaleD=1: 累加 (K 维迭代需要累积结果)
1891         WGMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMA_K,
1892                                     s_b + qidx * BK * BN + k_it * WGMMA_K,
1893                                     1, // ScaleD=1: accumulate (不清零)
1894                                     1, 1, 0, 0);
1895     }
1896 }
1897
1898 // Step C3: 提交并等待 WGMMA 完成
1899 // COMMIT_GROUP: 将此前发射的所有 WGMMA 指令归为一组提交。
1900 // WAIT_GROUP(0): 等待之前 commit 的所有 WGMMA 完成 (0 表示等待所有组)。
1901 // 注意 WGMMA 是异步执行 (async proxy), 这些指令用于同步 completion。
1902 WGMMA_COMMIT_GROUP();
1903 WGMMA_WAIT_GROUP(0);
1904
1905 // Step C4: 释放此 stage (发 empty 信号给 Producer)
1906 // 通知 Producer stage qidx 的 smem 已使用完毕, 可以安全覆盖。
1907 [[maybe_unused]] auto token = empty[qidx].arrive();
1908 }
1909
1910 // =====
1911 // Epilogue: 将寄存器中的累加结果写回 global memory C
1912 // =====
1913 //
1914 // WGMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
1915 //
1916 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
1917 // 这些 half 分布在 d[2][8][4] 数组中:
1918 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
1919 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
1920 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
1921 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
1922 //
1923 // 其中:
1924 //   row = warp * 16 + lane / 4    (0~63, 4 warps * 16 rows)
1925 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
1926 //
1927 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
1928 //   +-----+-----+
1929 //   | reg[0] | reg[2] | <- rows [row, row+8)
1930 //   |(col,+2)|(col+8)|
1931 //   +-----+-----+
1932 //   | reg[1] | reg[3] | <- rows [row+8, row+16)
1933 //   |(col,+2)|(col+8)|
1934 //   +-----+-----+
1935 //   每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
1936 //
1937 // 三维遍历:
1938 //   m_it=0: rows 0~63, m_it=1: rows 64~127
1939 //   g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
1940 //   每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
1941 //   128 线程 * 128 half = 16384 half = 128 * 128 ok
1942
1943 const int lane = tid % 32;
1944 const int warp = tid / 32;
1945 // row: 当前线程在当前 WGMMA atom 中负责的起始行
1946 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]

```

```

1947     const int row = warp * 16 + lane / 4;
1948     // block_C: 当前 C tile 在 global memory 中的起始地址
1949     // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
1950     half *block_C = C + by * BM * N + bx * BN;
1951
1952 #pragma unroll
1953     for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1954         int yo = m_it * WGMMMA_M; // M 方向行偏移 (0 或 64)
1955 #pragma unroll
1956         for (int g = 0; g < WGMMMA_N / 16; ++g) {
1957             int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
1958             // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
1959             // 左上象限: (row, col)
1960             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1961                 d[m_it][g][0];
1962             // 左下象限: (row+8, col)
1963             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1964                 d[m_it][g][1];
1965             // 右上象限: (row, col+8)
1966             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1967                 d[m_it][g][2];
1968             // 右下象限: (row+8, col+8)
1969             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1970                 d[m_it][g][3];
1971         }
1972     }
1973 }
1974 }
1975
1976 #if (defined(NOTES_V2_HAS_WGMMMA) \
1977     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
1978     || defined(__CUDA_ARCH_FEAT_SM90_ALL))
1979 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
1980 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
1981 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
1982 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
1983 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
1984 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
1985 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
1986 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
1987 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
1988 //
1989 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
1990
1991 template <int BlockMajorSize, int BlockMinorSize>
1992 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
1993         half *gmem_ptr,
1994         int blocks_height,
1995         int blocks_width) {
1996     void *gmem_address = (void *)gmem_ptr;
1997     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
1998         (uint64_t)BlockMajorSize * blocks_height,
1999         1, 1, 1};
2000     uint64_t gmem_prob_stride[5] = {
2001         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
2002     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
2003         uint32_t(BlockMajorSize), 1, 1, 1};
2004     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
2005     CUresult result = cuTensorMapEncodeTiled(
2006         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
2007         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
2008         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
2009         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);

```

```

2010 if (result != CUDA_SUCCESS)
2011     printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
2012 }
2013
2014 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
2015     half *src, int blocks_height, int blocks_width) {
2016     CUTensorMap *tma_map_d;
2017     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
2018     CUTensorMap tma_map_host;
2019     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
2020     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
2021         cudaMemcpyHostToDevice);
2022     return tma_map_d;
2023 }
2024 #endif /* NOTES_V2_HAS_WGMMMA */
2025
2026 // =====
2027 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
2028 // =====
2029 // 面试题点 (FlashAttention 算法) :
2030 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
2031 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
2032 // 2. FA 三板斧:
2033 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
2034 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
2035 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
2036 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
2037 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
2038 //    - 优点: 减少 warp 间通信和 shuffle
2039 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
2040 //    for each K,V tile:
2041 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
2042 //        m_new = max(m_old, row_max(S_cur * scale))
2043 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
2044 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
2045 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
2046 //        O_final = O_new / l_final
2047 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
2048 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
2049 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
2050 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
2051 //    - 这是此实现的寄存器布局复用技巧, 不要背成 “所有 MMA A/C fragment
2052 //      都天然同构” 的通用结论
2053 //
2054 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
2055 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
2056 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
2057 // Block: (128, 1, 1), kNumThreads=WARP_SIZE×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
2058 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
2059
2060 // ---- 寄存器填充辅助函数 ----
2061 template <typename T, int M, const int N, const int K = 2>
2062 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
2063     #pragma unroll
2064     for (int i = 0; i < M; ++i)
2065         #pragma unroll
2066         for (int j = 0; j < N; ++j)
2067             #pragma unroll
2068             for (int k = 0; k < K; ++k)
2069                 R[i][j][k] = val;
2070 }
2071
2072 template <typename T, int M, const int N = 2>

```

```

2073 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
2074 #pragma unroll
2075     for (int i = 0; i < M; ++i)
2076 #pragma unroll
2077         for (int j = 0; j < N; ++j)
2078             R[i][j] = val;
2079 }
2080
2081 // =====
2082 // FlashAttention-2 Split-Q Kernel (完整实现)
2083 // =====
2084 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
2085 //
2086 // Tile 设计 (以 kHeadDim=64 为例) :
2087 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
2088 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
2089 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
2090 //
2091 // 执行流程:
2092 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
2093 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
2094 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
2095 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
2096 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
2097 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
2098 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
2099 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
2100 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
2101 //   8) 最终 rescale: O_final = (1/l_final) * O_final
2102 //   9) Epilogue: warp shuffle + 128-bit collective store
2103
2104 template <
2105     const int kHeadDim,           // head dim: 32, 64, 128
2106     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
2107     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
2108     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
2109     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
2110     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
2111     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
2112     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
2113     const int kValTileSeqLenQ,   // value tiles along Q's M, 1 → Br per warp=16
2114     const int kValTileSeqLenK,   // value tiles along K's N, 8 → Bc_warp=8*8=64
2115     const int kValTileSeqLenP,   // value tiles for P@V M dim, 1
2116     const int kValTileHeadDimV,  // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
2117     const int kStage,            // pipeline stages for K: 1 or 2
2118     const int kPad>             // padding for bank conflict avoidance
2119 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
2120     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
2121                                     int QKV_seqlen, int QKV_head) {
2122
2123     // Tile dimensions
2124     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
2125     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
2126     constexpr int kNumThreads =
2127         WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
2128     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
2129     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
2130     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
2131     const float scale = 1.0f / sqrtf((float)kHeadDim);
2132
2133     // Block indexing
2134     const int QKV_batch_id = blockIdx.y / QKV_head;
2135     const int QKV_head_id = blockIdx.y % QKV_head;

```

```

2136 const int Q_tile_id = blockIdx.x;
2137 const int tid = threadIdx.x;
2138 const int warp_id = tid / WARP_SIZE;
2139 const int lane_id = tid % WARP_SIZE;
2140 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
2141 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
2142
2143 // Global memory base offsets for this (batch, head)
2144 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
2145 const int Q_gmem_offset =
2146     (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
2147     kHeadDim;
2148 const int K_gmem_offset = Q_gmem_offset;
2149 const int V_gmem_offset = Q_gmem_offset;
2150 const int O_gmem_offset = Q_gmem_offset;
2151
2152 // Thread-to-smem mapping for cooperative load
2153 int load_smem_Q_Br = tid / (kNumThreads / Br);
2154 int load_smem_Q_d =
2155     (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
2156 int load_smem_K_Bc = tid / (kNumThreads / Bc);
2157 int load_smem_K_d =
2158     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2159 int load_smem_V_Bc = tid / (kNumThreads / Bc);
2160 int load_smem_V_d =
2161     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
2162
2163 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
2164 if (load_gmem_Q_Br >= QKV_seqlen)
2165     return;
2166
2167 // ---- Shared memory layout ----
2168 extern __shared__ half smem[];
2169 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
2170 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
2171 half *Q_tile_smem = smem;
2172 half *K_tile_smem = Q_tile_smem + Q_tile_size;
2173 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
2174 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
2175 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
2176
2177 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
2178 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
2179 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
2180
2181 // ---- Online Softmax persistent state ----
2182 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
2183 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
2184 float lane_block_row_max_old[kValTileSeqLenQ][2];
2185 float lane_block_row_sum_old[kValTileSeqLenQ][2];
2186 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
2187 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
2188
2189 // ---- Register allocation ----
2190 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
2191 uint32_t R_K[kValTileSeqLenK][2]; // K regs
2192 uint32_t R_V[kValTileHeadDimV][2]; // V regs
2193 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
2194 // 对单个 m16n8k16 tile 而言:
2195 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
2196 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
2197 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
2198 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)

```

```

2199 uint32_t R_0[kValTileSeqLenP][kValTileHeadDimV][2]; // 0 for current tile
2200 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
2201         [2]; // 0 accumulator (final output)
2202
2203 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2204 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
2205
2206 // =====
2207 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
2208 // =====
2209 {
2210     int load_gmem_Q_addr =
2211         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
2212     uint32_t load_smem_Q_ptr =
2213         smem_Q_base_ptr +
2214         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
2215 #pragma unroll
2216     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
2217         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
2218     }
2219     CP_ASYNC_COMMIT_GROUP();
2220 }
2221
2222 // =====
2223 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
2224 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
2225 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
2226 // =====
2227 if constexpr (kStage > 1) {
2228 #pragma unroll
2229     for (int stage = 0; stage < (kStage - 1); ++stage) {
2230         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
2231         int load_gmem_K_addr =
2232             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2233         uint32_t load_smem_K_ptr =
2234             smem_K_base_ptr +
2235             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2236              load_smem_K_d) *
2237             sizeof(half);
2238 #pragma unroll
2239         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2240             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2241         }
2242         CP_ASYNC_COMMIT_GROUP();
2243     }
2244     CP_ASYNC_WAIT_GROUP(kStage - 2);
2245     __syncthreads();
2246 }
2247
2248 // =====
2249 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
2250 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
2251 // =====
2252 #pragma unroll 1
2253 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
2254     int smem_sel = tile_K_seqlen % kStage;
2255     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
2256
2257     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
2258     if constexpr (kStage > 1) {
2259         // 只有 kStage>1 才能真正做 K 的 pipeline:
2260         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
2261         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。

```



```

2262 // Load current V tile (no pipeline for V — one stage is enough)
2263 {
2264     int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
2265     int load_gmem_V_addr =
2266         V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2267     uint32_t load_smem_V_ptr =
2268         smem_V_base_ptr +
2269         (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2270 #pragma unroll
2271     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2272         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2273     }
2274     CP_ASYNC_COMMIT_GROUP();
2275 }
2276
2277 // Prefetch next K tile (pipelined)
2278 if ((tile_K_seqlen + 1) < Tc) {
2279     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2280     int load_gmem_K_addr =
2281         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2282     uint32_t load_smem_K_ptr =
2283         smem_K_base_ptr +
2284         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2285          load_smem_K_d) *
2286         sizeof(half);
2287 #pragma unroll
2288     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2289         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2290     }
2291     CP_ASYNC_COMMIT_GROUP();
2292 }
2293 }
2294
2295 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2296 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
2297 #pragma unroll
2298 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2299     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2300 #pragma unroll
2301     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2302         int warp_smem_Q_Br =
2303             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
2304         int lane_smem_Q_Br =
2305             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2306         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2307         uint32_t lane_smem_Q_ptr =
2308             smem_Q_base_ptr +
2309             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2310         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2311                    lane_smem_Q_ptr);
2312     }
2313
2314     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2315     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2316 #pragma unroll
2317     for (int j = 0; j < kValTileSeqLenK; ++j) {
2318         int warp_smem_K_Bc =
2319             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
2320         int lane_smem_K_Bc =
2321             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2322         int lane_smem_K_d =
2323             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2324         uint32_t lane_smem_K_ptr =

```



```

2325         smem_K_base_ptr +
2326         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2327         lane_smem_K_d) *
2328         sizeof(half);
2329         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2330     }
2331
2332     // MMA: S[tile] += Q[tile] @ K^T[tile]
2333 #pragma unroll
2334     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2335 #pragma unroll
2336         for (int j = 0; j < kValTileSeqLenK; ++j) {
2337             MMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2338                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2339                 R_S[i][j][1]);
2340         }
2341     }
2342 } // end loop over d
2343 __syncthreads();
2344
2345 // =====
2346 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to
2347 // R_S
2348 // =====
2349 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2350 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2351 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2352 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2353 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2354 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2355 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2356 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
2357 // kValTileSeqLenK tiles.
2358
2359 float lane_row_max_new[kValTileSeqLenQ][2];
2360 float lane_row_sum_new[kValTileSeqLenQ][2];
2361 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2362 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2363
2364 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
2365 #pragma unroll
2366     for (int i = 0; i < kValTileSeqLenQ; ++i) {
2367 #pragma unroll
2368         for (int j = 0; j < kValTileSeqLenK; ++j) {
2369             // Extract half values from R_S registers (C matrix fragment layout)
2370             float2 t_reg_S_0 =
2371                 __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2372             float2 t_reg_S_1 =
2373                 __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2374             // S = (Q@K^T) * scale
2375             float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2376             float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2377             lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2378             lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2379         }
2380         // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2381         // {0,4,8,...,28} hold valid data)
2382         lane_row_max_new[i][0] =
2383             warp_reduce_max<4, float>(lane_row_max_new[i][0]);
2384         lane_row_max_new[i][1] =
2385             warp_reduce_max<4, float>(lane_row_max_new[i][1]);
2386     }
2387

```

```

2388 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
2389 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2390 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2391 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2392 #pragma unroll
2393 for (int i = 0; i < kValTileSeqLenQ; ++i) {
2394     // m_new = max(m_old, m_cur)
2395     float block_row_max_new_0 =
2396         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2397     float block_row_max_new_1 =
2398         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2399
2400 #pragma unroll
2401 for (int j = 0; j < kValTileSeqLenK; ++j) {
2402     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2403     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2404
2405     // P = exp(S * scale - m_new), 用 fma 保证精度
2406     t_reg_S_0.x =
2407         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2408     t_reg_S_0.y =
2409         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2410     t_reg_S_1.x =
2411         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2412     t_reg_S_1.y =
2413         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2414
2415     // Accumulate row sums
2416     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2417     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2418
2419     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2420     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2421     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2422 }
2423
2424 // Warp-level reduce sum (warp_size = 4, same as max)
2425 lane_row_sum_new[i][0] =
2426     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
2427 lane_row_sum_new[i][1] =
2428     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
2429 }
2430 __syncthreads();
2431
2432 // =====
2433 // 3d: P@V - P[Br,Bc] @ V[Bc,d] = 0[Br,d]
2434 // =====
2435 // Wait for V to be ready before computing P@V
2436 if constexpr (kStage > 1) {
2437     if ((tile_K_seqLen + 1) < Tc) {
2438         CP_ASYNC_WAIT_GROUP(1);
2439     } else {
2440         CP_ASYNC_WAIT_GROUP(0);
2441     }
2442 } else {
2443     CP_ASYNC_WAIT_GROUP(0);
2444 }
2445 __syncthreads();
2446
2447 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
2448
2449 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2450 // Bc=kMmaAtomK=16 -> 1 iteration for kHeadDim≤64 configurations

```

```

2451 #pragma unroll
2452 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2453     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2454     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2455     // transposed ldmatrix
2456 #pragma unroll
2457 for (int j = 0; j < kValTileHeadDimV; ++j) {
2458     int warp_smem_V_d =
2459         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2460     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2461     int lane_smem_V_d = warp_smem_V_d;
2462     uint32_t lane_smem_V_ptr =
2463         smem_V_base_ptr +
2464         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2465     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2466 }
2467
2468 // P matrix layout in R_S[i][j][2]:
2469 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2470 // | 64x64 | warp_KV 0 |
2471 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2472 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2473 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2474 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2475 // tile_V_Bc selects which 16-column slice of P to use:
2476 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2477 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2478 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2479 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2480 // 对应的 MMA A fragment 布局可以这样记:
2481 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2482 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2483 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2484 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2485 // fragment 都无条件成立的通用结论。layout转换逻辑:
2486 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2487 int w = tile_V_Bc * 2;
2488 #pragma unroll
2489 for (int i = 0; i < kValTileSeqLenP; ++i) {
2490 #pragma unroll
2491 for (int j = 0; j < kValTileHeadDimV; ++j) {
2492     HMMA16816(R_O[i][j][0], R_O[i][j][1], // C fragment output
2493         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2494         R_V[j][0], R_V[j][1], // B fragment = V
2495         R_O[i][j][0], R_O[i][j][1]); // C fragment output
2496     }
2497 }
2498 } // end for tile_V_Bc
2499 __syncthreads();
2500
2501 // =====
2502 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2503 // =====
2504 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
2505 #pragma unroll
2506 for (int i = 0; i < kValTileSeqLenP; ++i) {
2507     float block_row_max_new_0 = lane_row_max_new[i][0];
2508     float block_row_max_new_1 = lane_row_max_new[i][1];
2509     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2510     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2511
2512     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2513     float block_row_max_old_1 = lane_block_row_max_old[i][1];

```

```

2514
2515     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2516     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2517
2518     // Handle first iteration: m_old = -inf, need to use m_new directly
2519     block_row_max_old_0 =
2520         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2521     block_row_max_old_1 =
2522         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2523
2524     float rescale_o_factor_0 =
2525         __expf(block_row_max_old_0 - block_row_max_new_0);
2526     float rescale_o_factor_1 =
2527         __expf(block_row_max_old_1 - block_row_max_new_1);
2528
2529     // Rescale O_old + Add P@V in one fused step
2530 #pragma unroll
2531     for (int j = 0; j < kValTileHeadDimV; ++j) {
2532         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2533         //   reg[0] -> rows 0~7 的 {c0,c1}
2534         //   reg[1] -> rows 8~15 的 {c2,c3}
2535         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2536         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2537         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2538         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2539
2540         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
2541         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2542         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2543         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2544         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2545
2546         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2547         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2548     }
2549
2550     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2551     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2552     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2553     lane_block_row_sum_old[i][0] = __fmaf_rn(
2554         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2555     lane_block_row_sum_old[i][1] = __fmaf_rn(
2556         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2557
2558     // Update m
2559     lane_block_row_max_old[i][0] = block_row_max_new_0;
2560     lane_block_row_max_old[i][1] = block_row_max_new_1;
2561 }
2562
2563 // Wait for next K tile to be ready in smem before next iteration
2564 if constexpr (kStage > 1) {
2565     if ((tile_K_seqlen + 1) < Tc) {
2566         CP_ASYNC_WAIT_GROUP(0);
2567     }
2568     __syncthreads();
2569 }
2570 } // end outer loop over K seqlen
2571 __syncthreads();
2572
2573 // =====
2574 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
2575 // =====
2576 #pragma unroll

```

```

2577     for (int i = 0; i < kValTileSeqLenP; ++i) {
2578         float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2579         float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2580 #pragma unroll
2581         for (int j = 0; j < kValTileHeadDimV; ++j) {
2582             float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2583             float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2584             t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2585             t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2586             t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2587             t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2588             HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2589             HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2590         }
2591     }
2592
2593     // =====
2594     // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2595     // =====
2596     // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2597     // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2598 #pragma unroll
2599     for (int i = 0; i < kValTileSeqLenP; ++i) {
2600 #pragma unroll
2601         for (int j = 0; j < kValTileHeadDimV; ++j) {
2602             uint32_t R_Z[2][4];
2603             R_Z[0][0] = R_D[i][j][0];
2604             R_Z[1][0] = R_D[i][j][1];
2605             R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2606             R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2607             R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2608             R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2609             R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2610             R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2611
2612             // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2613             if (lane_id % 4 == 0) {
2614                 int store_warp_regs_0_Br =
2615                     warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
2616                 int store_lane_gmem_0_Br =
2617                     Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2618                 int store_warp_regs_0_d =
2619                     warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
2620                 int store_lane_gmem_0_d = store_warp_regs_0_d;
2621                 int store_gmem_0_addr_0 = 0_gmem_offset +
2622                     (store_lane_gmem_0_Br + 0) * kHeadDim +
2623                     store_lane_gmem_0_d;
2624                 int store_gmem_0_addr_1 = 0_gmem_offset +
2625                     (store_lane_gmem_0_Br + 8) * kHeadDim +
2626                     store_lane_gmem_0_d;
2627                 // LDST128BITS = reinterpret_cast<float4*>
2628                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2629                     *reinterpret_cast<float4*>(&R_Z[0][0]);
2630                 *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2631                     *reinterpret_cast<float4*>(&R_Z[1][0]);
2632             }
2633         }
2634     }
2635 }
2636
2637 // =====
2638 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
2639 // =====

```

```

2640
2641 static inline void check(cudaError_t err, const char *msg) {
2642     if (err != cudaSuccess) {
2643         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
2644         exit(EXIT_FAILURE);
2645     }
2646 }
2647
2648
2649 static void test_block_reduce(int N) {
2650
2651     srand(42);
2652     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2653     for (int i = 0; i < N; i++)
2654         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2655
2656     // CPU reference: sum of all elements
2657     double ref = 0.0;
2658     for (int i = 0; i < N; i++) ref += (double)h_a[i];
2659
2660     float *d_a, *d_y;
2661     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
2662     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
2663
2664     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
2665     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
2666
2667     dim3 block(128);
2668     dim3 grid((N + 127) / 128);
2669     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
2670     check(cudaGetLastError(), "blockreduce launch");
2671     check(cudaDeviceSynchronize(), "blockreduce sync");
2672
2673     float result;
2674     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
2675
2676     float err = fabsf(result - (float)ref);
2677     printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
2678         err < 1e-2f ? "PASS" : "FAIL");
2679
2680     free(h_a);
2681     cudaFree(d_a); cudaFree(d_y);
2682 }
2683
2684
2685 static void test_dot(int N) {
2686
2687     srand(42);
2688     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2689     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2690     for (int i = 0; i < N; i++) {
2691         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2692         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2693     }
2694
2695     // CPU reference
2696     double ref = 0.0;
2697     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
2698
2699     float *d_a, *d_b, *d_y;
2700     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
2701     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
2702     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");

```

```

2703
2704 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
2705 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
2706 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
2707
2708 dim3 block(128);
2709 dim3 grid((N + 127) / 128);
2710 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
2711 check(cudaGetLastError(), "dot launch");
2712 check(cudaDeviceSynchronize(), "dot sync");
2713
2714 float result;
2715 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
2716
2717 float err = fabsf(result - (float)ref);
2718 printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
2719     err < 1e-2f ? "PASS" : "FAIL");
2720
2721 // ---- Dot Vec4 ----
2722 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
2723 dim3 block_v4(32);
2724 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
2725 check(cudaGetLastError(), "dot_vec4 launch");
2726 check(cudaDeviceSynchronize(), "dot_vec4 sync");
2727
2728 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
2729 float err_v4 = fabsf(result - (float)ref);
2730 printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
2731
2732 free(h_a); free(h_b);
2733 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
2734 }
2735
2736
2737 static void test_relu(int N) {
2738     srand(42);
2739     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2740     float *h_y = (float *)malloc((size_t)N * sizeof(float));
2741     for (int i = 0; i < N; i++)
2742         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2743
2744     float *d_x, *d_y;
2745     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
2746     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
2747     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
2748
2749     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
2750     dim3 block256(256);
2751     dim3 grid256((N + 255) / 256);
2752     relu<<<grid256, block256>>>(d_x, d_y, N);
2753     check(cudaGetLastError(), "relu launch");
2754     check(cudaDeviceSynchronize(), "relu sync");
2755     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
2756     float max_err = 0.0f;
2757     for (int i = 0; i < N; i++) {
2758         float expected = fmaxf(0.0f, h_x[i]);
2759         float err = fabsf(h_y[i] - expected);
2760         if (err > max_err) max_err = err;
2761     }
2762     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2763
2764     dim3 block64(64);
2765     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);

```



```

2766 check(cudaGetLastError(), "relu_vec4 launch");
2767 check(cudaDeviceSynchronize(), "relu_vec4 sync");
2768 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
2769 max_err = 0.0f;
2770 for (int i = 0; i < N; i++) {
2771     float expected = fmaxf(0.0f, h_x[i]);
2772     float err = fabsf(h_y[i] - expected);
2773     if (err > max_err) max_err = err;
2774 }
2775 printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2776
2777 free(h_x); free(h_y);
2778 cudaFree(d_x); cudaFree(d_y);
2779 }
2780
2781
2782 static void test_elementwise(int N) {
2783     srand(42);
2784     float *h_a = (float *)malloc((size_t)N * sizeof(float));
2785     float *h_b = (float *)malloc((size_t)N * sizeof(float));
2786     for (int i = 0; i < N; i++) {
2787         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2788         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2789     }
2790
2791     float *d_a, *d_b, *d_c;
2792     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
2793     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
2794     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
2795     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
2796     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
2797
2798     dim3 block256(256);
2799     dim3 grid256((N + 255) / 256);
2800     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
2801     check(cudaGetLastError(), "eadd launch");
2802     check(cudaDeviceSynchronize(), "eadd sync");
2803     float *h_c = (float *)malloc((size_t)N * sizeof(float));
2804     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
2805     float max_err = 0.0f;
2806     for (int i = 0; i < N; i++) {
2807         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2808         if (err > max_err) max_err = err;
2809     }
2810     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2811
2812     dim3 block64(64);
2813     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
2814     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
2815     check(cudaGetLastError(), "eadd_vec4 launch");
2816     check(cudaDeviceSynchronize(), "eadd_vec4 sync");
2817     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
2818     max_err = 0.0f;
2819     for (int i = 0; i < N; i++) {
2820         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
2821         if (err > max_err) max_err = err;
2822     }
2823     printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2824
2825     free(h_a); free(h_b); free(h_c);
2826     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
2827 }
2828

```

```

2829
2830 static void test_histogram(int N) {
2831     srand(42);
2832     int BINS = 16;
2833     int *h_hist = (int *)calloc(BINS, sizeof(int));
2834     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
2835     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
2836     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
2837     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
2838
2839     int *d_idx, *d_hist;
2840     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
2841     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
2842     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
2843     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
2844
2845     dim3 block256(256);
2846     dim3 grid256((N + 255) / 256);
2847     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
2848     check(cudaGetLastError(), "histogram launch");
2849     check(cudaDeviceSynchronize(), "histogram sync");
2850     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
2851
2852     float max_err = 0.0f;
2853     for (int i = 0; i < BINS; i++) {
2854         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
2855         if (err > max_err) max_err = err;
2856     }
2857     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2858
2859     free(h_hist); free(h_hist_ref); free(h_idx);
2860     cudaFree(d_idx); cudaFree(d_hist);
2861 }
2862
2863
2864 static void test_softmax(int N) {
2865     // Use N = blockDim.x so one block processes all elements as one token.
2866     constexpr int NUM_THREADS = 256;
2867     if (N != NUM_THREADS) {
2868         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", NUM_THREADS, N);
2869         return;
2870     }
2871
2872     srand(42);
2873     float *h_x = (float *)malloc((size_t)N * sizeof(float));
2874     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
2875     for (int i = 0; i < N; i++)
2876         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
2877
2878     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
2879     float max_val = -FLT_MAX;
2880     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
2881     double sum_exp = 0.0;
2882     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
2883     for (int i = 0; i < N; i++)
2884         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
2885
2886     float *d_x, *d_y;
2887     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
2888     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
2889
2890     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
2891

```

```

2892 dim3 block(256);
2893 dim3 grid(1); // one token covering all N elements
2894 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2895 check(cudaGetLastError(), "softmax launch");
2896 check(cudaDeviceSynchronize(), "softmax sync");
2897
2898 float *h_y = (float *)malloc((size_t)N * sizeof(float));
2899 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
2900
2901 float max_err = 0.0f;
2902 for (int i = 0; i < N; i++) {
2903     float err = fabsf(h_y[i] - h_y_ref[i]);
2904     if (err > max_err) max_err = err;
2905 }
2906 printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2907
2908 // ---- Safe Softmax ----
2909 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2910 check(cudaGetLastError(), "safe_softmax launch");
2911 check(cudaDeviceSynchronize(), "safe_softmax sync");
2912 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
2913 max_err = 0.0f;
2914 for (int i = 0; i < N; i++) {
2915     float err = fabsf(h_y[i] - h_y_ref[i]);
2916     if (err > max_err) max_err = err;
2917 }
2918 printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2919
2920 // ---- Naive Softmax ----
2921 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
2922 check(cudaGetLastError(), "naive_softmax launch");
2923 check(cudaDeviceSynchronize(), "naive_softmax sync");
2924 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
2925 max_err = 0.0f;
2926 for (int i = 0; i < N; i++) {
2927     float err = fabsf(h_y[i] - h_y_ref[i]);
2928     if (err > max_err) max_err = err;
2929 }
2930 printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2931
2932 free(h_x); free(h_y); free(h_y_ref);
2933 cudaFree(d_x); cudaFree(d_y);
2934 }
2935
2936
2937 static void test_rms_norm(int N, int K) {
2938
2939     srand(42);
2940     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2941     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
2942     for (int i = 0; i < N * K; i++)
2943         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
2944     float g = 1.5f; // gain
2945
2946     // CPU reference: y = (x / rms(x)) * g
2947     float epsilon = 1e-5f;
2948     for (int n = 0; n < N; n++) {
2949         double sum_sq = 0.0;
2950         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
2951         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
2952         for (int k = 0; k < K; k++)
2953             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
2954     }

```

```

2955 float *d_x, *d_y;
2956 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
2957 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
2958 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
2960
2961 dim3 block(128);
2962 dim3 grid(N);
2963 rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
2964 check(cudaGetLastError(), "rmsnorm launch");
2965 check(cudaDeviceSynchronize(), "rmsnorm sync");
2966
2967 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
2968 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
2969
2970 float max_err = 0.0f;
2971 for (int i = 0; i < N * K; i++) {
2972     float err = fabsf(h_y[i] - h_y_ref[i]);
2973     if (err > max_err) max_err = err;
2974 }
2975 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2976
2977 // ---- RMS Norm Vec4 ----
2978 dim3 block_rv4(32);
2979 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
2980 check(cudaGetLastError(), "rmsnorm_vec4 launch");
2981 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
2982 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
2983 max_err = 0.0f;
2984 for (int i = 0; i < N * K; i++) {
2985     float err = fabsf(h_y[i] - h_y_ref[i]);
2986     if (err > max_err) max_err = err;
2987 }
2988 printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
2989
2990 free(h_x); free(h_y); free(h_y_ref);
2991 cudaFree(d_x); cudaFree(d_y);
2992 }
2993
2994
2995 static void test_layer_norm(int N, int K) {
2996
2997     srand(42);
2998     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
2999     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
3000     for (int i = 0; i < N * K; i++)
3001         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3002     float g = 1.5f, b = 0.3f; // gain and bias
3003
3004     // CPU reference: y = ((x - mean) / std) * g + b
3005     float epsilon = 1e-5f;
3006     for (int n = 0; n < N; n++) {
3007         double sum = 0.0;
3008         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
3009         float mean = (float)sum / (float)K;
3010         double sum_sq = 0.0;
3011         for (int k = 0; k < K; k++) {
3012             float diff = h_x[n * K + k] - mean;
3013             sum_sq += (double)diff * (double)diff;
3014         }
3015         float std = sqrtf((float)sum_sq / (float)K + epsilon);
3016         for (int k = 0; k < K; k++)
3017             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;

```

```

3018 }
3019
3020 float *d_x, *d_y;
3021 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
3022 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
3023 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
3024
3025 dim3 block(128);
3026 dim3 grid(N);
3027 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
3028 check(cudaGetLastError(), "layernorm launch");
3029 check(cudaDeviceSynchronize(), "layernorm sync");
3030
3031 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
3032 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
3033
3034 float max_err = 0.0f;
3035 for (int i = 0; i < N * K; i++) {
3036     float err = fabsf(h_y[i] - h_y_ref[i]);
3037     if (err > max_err) max_err = err;
3038 }
3039 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3040
3041 // ---- Layer Norm Vec4 ----
3042 dim3 block_lv4(32);
3043 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
3044 check(cudaGetLastError(), "layernorm_vec4 launch");
3045 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
3046 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
3047 max_err = 0.0f;
3048 for (int i = 0; i < N * K; i++) {
3049     float err = fabsf(h_y[i] - h_y_ref[i]);
3050     if (err > max_err) max_err = err;
3051 }
3052 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3053
3054 free(h_x); free(h_y); free(h_y_ref);
3055 cudaFree(d_x); cudaFree(d_y);
3056 }
3057
3058
3059 static void test_rope(int seq_len, int N) {
3060
3061     int total_pairs = seq_len * N;
3062     int total_elems = total_pairs * 2;
3063     size_t size = (size_t)total_elems * sizeof(float);
3064
3065     float *h_x = (float *)malloc(size);
3066     float *h_y_ref = (float *)malloc(size);
3067
3068     srand(42);
3069     for (int i = 0; i < total_elems; i++)
3070         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3071
3072     // CPU reference: 2D rotation for each pair
3073     for (int idx = 0; idx < total_pairs; idx++) {
3074         int token_pos = idx / N;
3075         int token_idx = idx % N;
3076         float x1 = h_x[idx * 2];
3077         float x2 = h_x[idx * 2 + 1];
3078         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
3079         float angle = (float)token_pos * theta;
3080         float cos_v = cosf(angle);

```

```

3081     float sin_v = sinf(angle);
3082     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
3083     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
3084 }
3085
3086 float *d_x, *d_y;
3087 check(cudaMalloc(&d_x, size), "rope alloc X");
3088 check(cudaMalloc(&d_y, size), "rope alloc Y");
3089 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
3090
3091 dim3 block(256);
3092 dim3 grid((total_pairs + 255) / 256);
3093 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
3094 check(cudaGetLastError(), "rope launch");
3095 check(cudaDeviceSynchronize(), "rope sync");
3096
3097 float *h_y = (float *)malloc(size);
3098 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
3099
3100 float max_err = 0.0f;
3101 for (int i = 0; i < total_elems; i++) {
3102     float err = fabsf(h_y[i] - h_y_ref[i]);
3103     if (err > max_err) max_err = err;
3104 }
3105 printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
3106
3107 free(h_x);
3108 free(h_y);
3109 free(h_y_ref);
3110 cudaFree(d_x);
3111 cudaFree(d_y);
3112 }
3113
3114
3115 static void test_mat_transpose(int row, int col) {
3116
3117     size_t size_in = (size_t)row * col * sizeof(float);
3118     size_t size_out = (size_t)col * row * sizeof(float);
3119
3120     float *h_x = (float *)malloc(size_in);
3121     float *h_y_ref = (float *)malloc(size_out);
3122
3123     srand(42);
3124     for (int i = 0; i < row * col; i++)
3125         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3126
3127     // CPU reference: y[j][i] = x[i][j] (row-major)
3128     for (int i = 0; i < row; i++)
3129         for (int j = 0; j < col; j++)
3130             h_y_ref[j * row + i] = h_x[i * col + j];
3131
3132     float *d_x, *d_y;
3133     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
3134     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
3135     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
3136
3137     dim3 block(16, 16);
3138     dim3 grid((col + 15) / 16, (row + 15) / 16);
3139     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
3140     check(cudaGetLastError(), "mattrans launch");
3141     check(cudaDeviceSynchronize(), "mattrans sync");
3142
3143     float *h_y = (float *)malloc(size_out);

```

```

3144 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
3145
3146 float max_err = 0.0f;
3147 for (int i = 0; i < col * row; i++) {
3148     float err = fabsf(h_y[i] - h_y_ref[i]);
3149     if (err > max_err) max_err = err;
3150 }
3151 printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3152
3153 free(h_x);
3154 free(h_y);
3155 free(h_y_ref);
3156 cudaFree(d_x);
3157 cudaFree(d_y);
3158 }
3159
3160
3161 static void test_mat_transpose_padded(int row, int col) {
3162
3163     size_t size_in = (size_t)row * col * sizeof(float);
3164     size_t size_out = (size_t)col * row * sizeof(float);
3165
3166     float *h_x = (float *)malloc(size_in);
3167     float *h_y_ref = (float *)malloc(size_out);
3168
3169     srand(42);
3170     for (int i = 0; i < row * col; i++)
3171         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3172
3173     // CPU reference: y[j][i] = x[i][j] (row-major)
3174     for (int i = 0; i < row; i++)
3175         for (int j = 0; j < col; j++)
3176             h_y_ref[j * row + i] = h_x[i * col + j];
3177
3178     float *d_x, *d_y;
3179     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
3180     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
3181     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
3182
3183     dim3 block(16, 16);
3184     dim3 grid((col + 15) / 16, (row + 63) / 64);
3185     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
3186     check(cudaGetLastError(), "mattrans_padded launch");
3187     check(cudaDeviceSynchronize(), "mattrans_padded sync");
3188
3189     float *h_y = (float *)malloc(size_out);
3190     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
3191
3192     float max_err = 0.0f;
3193     for (int i = 0; i < col * row; i++) {
3194         float err = fabsf(h_y[i] - h_y_ref[i]);
3195         if (err > max_err) max_err = err;
3196     }
3197     printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
3198
3199     free(h_x);
3200     free(h_y);
3201     free(h_y_ref);
3202     cudaFree(d_x);
3203     cudaFree(d_y);
3204 }
3205
3206

```



```

3207 static void test_sgemv(int M, int K) {
3208
3209     srand(42);
3210     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
3211     float *h_x = (float *)malloc((size_t)K * sizeof(float));
3212     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
3213     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3214     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3215
3216     // CPU reference: y[m] = sum_k A[m,k] * x[k]
3217     for (int m = 0; m < M; m++) {
3218         double sum = 0.0;
3219         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
3220         h_y_ref[m] = (float)sum;
3221     }
3222
3223     float *d_a, *d_x, *d_y;
3224     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
3225     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
3226     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
3227
3228     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
3229     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
3230
3231     dim3 block(32, 4);
3232     dim3 grid((M + 3) / 4);
3233     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
3234     check(cudaGetLastError(), "sgemv launch");
3235     check(cudaDeviceSynchronize(), "sgemv sync");
3236
3237     float *h_y = (float *)malloc((size_t)M * sizeof(float));
3238     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
3239
3240     float max_err = 0.0f;
3241     for (int m = 0; m < M; m++) {
3242         float err = fabsf(h_y[m] - h_y_ref[m]);
3243         if (err > max_err) max_err = err;
3244     }
3245     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3246
3247     // ---- SGEMV K32 ----
3248     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
3249     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
3250     check(cudaGetLastError(), "sgemv_k32 launch");
3251     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
3252
3253     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
3254
3255     max_err = 0.0f;
3256     for (int m = 0; m < M; m++) {
3257         float err = fabsf(h_y[m] - h_y_ref[m]);
3258         if (err > max_err) max_err = err;
3259     }
3260     printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3261
3262     // ---- SGEMV K16 ----
3263     free(h_a); free(h_x); free(h_y); free(h_y_ref);
3264     cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3265
3266     int K16 = 16;
3267     h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
3268     h_x = (float *)malloc((size_t)K16 * sizeof(float));
3269     h_y_ref = (float *)malloc((size_t)M * sizeof(float));

```

```

3270 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3271 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3272
3273 for (int m = 0; m < M; m++) {
3274     double sum = 0.0;
3275     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
3276     h_y_ref[m] = (float)sum;
3277 }
3278
3279 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
3280 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
3281 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
3282
3283 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
3284 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
3285
3286 dim3 grid_k16((M + 7) / 8);
3287 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
3288 check(cudaGetLastError(), "sgemv_k16 launch");
3289 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
3290
3291 h_y = (float *)malloc((size_t)M * sizeof(float));
3292 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
3293
3294 max_err = 0.0f;
3295 for (int m = 0; m < M; m++) {
3296     float err = fabsf(h_y[m] - h_y_ref[m]);
3297     if (err > max_err) max_err = err;
3298 }
3299 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3300
3301 free(h_a); free(h_x); free(h_y); free(h_y_ref);
3302 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
3303 }
3304
3305
3306 static void test_sgemm(int M, int N, int K) {
3307
3308     size_t size_a = (size_t)M * K * sizeof(float);
3309     size_t size_b = (size_t)K * N * sizeof(float);
3310     size_t size_c = (size_t)M * N * sizeof(float);
3311
3312     float *h_a = (float *)malloc(size_a);
3313     float *h_b = (float *)malloc(size_b);
3314     float *h_c_ref = (float *)malloc(size_c);
3315
3316     srand(42);
3317     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3318     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
3319
3320     float *d_a, *d_b, *d_c;
3321     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
3322     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
3323     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
3324
3325     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
3326     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
3327
3328     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
3329     cublasHandle_t handle;
3330     cublasCreate(&handle);
3331     float alpha = 1.0f, beta = 0.0f;
3332     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,

```

```

3333         &alpha, d_b, N, d_a, K, &beta, d_c, N);
3334 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
3335
3336 // Kernel
3337 cudaEvent_t start, stop;
3338 cudaEventCreate(&start);
3339 cudaEventCreate(&stop);
3340
3341 dim3 block(32, 32);
3342 dim3 grid((N + 31) / 32, (M + 31) / 32);
3343 sgemmm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
3344 check(cudaGetLastError(), "sgemm launch");
3345 check(cudaDeviceSynchronize(), "sgemm sync");
3346
3347 float *h_c = (float *)malloc(size_c);
3348 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
3349
3350 // Verify
3351 float max_err = 0.0f;
3352 for (int i = 0; i < M * N; i++) {
3353     float err = fabsf(h_c[i] - h_c_ref[i]);
3354     if (err > max_err) max_err = err;
3355 }
3356 printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3357
3358 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
3359
3360 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
3361 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
3362 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
3363 check(cudaGetLastError(), "sgemm_vec4 launch");
3364 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
3365
3366 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
3367
3368 max_err = 0.0f;
3369 for (int i = 0; i < M * N; i++) {
3370     float err = fabsf(h_c[i] - h_c_ref[i]);
3371     if (err > max_err) max_err = err;
3372 }
3373 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
3374
3375 free(h_a); free(h_b); free(h_c); free(h_c_ref);
3376 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
3377 cublasDestroy(handle);
3378 cudaEventDestroy(start);
3379 cudaEventDestroy(stop);
3380 }
3381
3382
3383 static void test_hgemm_mma(int M, int N, int K) {
3384
3385     size_t size_a = (size_t)M * K * sizeof(half);
3386     size_t size_b = (size_t)K * N * sizeof(half);
3387     size_t size_c = (size_t)M * N * sizeof(half);
3388
3389     half *h_a = (half *)malloc(size_a);
3390     half *h_b = (half *)malloc(size_b);
3391     half *h_c_ref = (half *)malloc(size_c);
3392
3393     srand(42);
3394     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3395     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);

```

```

3396 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
3397 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
3398 size_t size_b_t = (size_t)N * K * sizeof(half);
3399 half *h_b_t = (half *)malloc(size_b_t);
3400 for (int n = 0; n < N; n++)
3401     for (int k = 0; k < K; k++)
3402         h_b_t[n * K + k] = h_b[k * N + n];
3403
3404
3405 half *d_a, *d_b, *d_b_t, *d_c;
3406 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
3407 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
3408 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
3409 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
3410
3411 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
3412 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
3413 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
3414
3415 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
3416 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
3417 cublasHandle_t handle;
3418 cublasCreate(&handle);
3419 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3420 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
3421             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
3422             &beta_h, d_c, CUDA_R_16F, N,
3423             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3424 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
3425
3426 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
3427 cudaEvent_t start, stop;
3428 cudaEventCreate(&start);
3429 cudaEventCreate(&stop);
3430
3431 constexpr int BM = 128, BN = 128, BK = 16, K_STAGE = 3;
3432 size_t smem_bytes = K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 24576
3433 dim3 block(256);
3434 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3435 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
3436 check(cudaGetLastError(), "hgemm launch");
3437 check(cudaDeviceSynchronize(), "hgemm sync");
3438
3439 half *h_c = (half *)malloc(size_c);
3440 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
3441
3442 // Verify
3443 float max_err = 0.0f;
3444 for (int i = 0; i < M * N; i++) {
3445     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3446     if (err > max_err) max_err = err;
3447 }
3448 printf("| %-35s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
3449
3450 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
3451 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
3452 cublasDestroy(handle);
3453 cudaEventDestroy(start);
3454 cudaEventDestroy(stop);
3455 }
3456
3457 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(

```

```

__CUDA_ARCH_FEAT_SM90_ALL)
3459 static void test_hgemm_wgmma(int M, int N, int K) {
3460     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
3461     // TN layout: C[M×N] = A[M×K] × BT[N×K]
3462     // Kernel: hgemm_wgmma_stages_tn with default template params
3463
3464     constexpr int BM = 128, BN = 128, BK = 64, K_STAGE = 3, NUM_THREADS = 256;
3465
3466     // M, K must be divisible by tile dims
3467     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
3468         printf("| %-35s | %-12s | %-4s |\n",
3469             "HGEMM WGMMMA", "SKIP", "SKIP");
3470         return;
3471     }
3472
3473     size_t size_a = (size_t)M * K * sizeof(half);
3474     size_t size_b = (size_t)K * N * sizeof(half);
3475     size_t size_c = (size_t)M * N * sizeof(half);
3476
3477     half *h_a = (half *)malloc(size_a);
3478     half *h_b = (half *)malloc(size_b);
3479     half *h_c_ref = (half *)malloc(size_c);
3480
3481     srand(42);
3482     for (int i = 0; i < M * K; i++)
3483         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3484     for (int i = 0; i < K * N; i++)
3485         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3486
3487     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
3488     size_t size_b_t = (size_t)N * K * sizeof(half);
3489     half *h_b_t = (half *)malloc(size_b_t);
3490     for (int n = 0; n < N; n++)
3491         for (int k = 0; k < K; k++)
3492             h_b_t[n * K + k] = h_b[k * N + n];
3493
3494     half *d_a, *d_b, *d_b_t, *d_c;
3495     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
3496     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
3497     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
3498     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
3499
3500     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
3501     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
3502         "wgmma H2D B (cuBLAS)");
3503     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
3504         "wgmma H2D B_t (kernel)");
3505
3506     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
3507     cublasHandle_t handle;
3508     cublasCreate(&handle);
3509     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
3510     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
3511         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
3512         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
3513     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
3514         "wgmma D2H ref");
3515
3516     // Create TMA tensor maps for A and BT
3517     // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
3518     // BT[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
3519     CUtensorMap *tma_a =
3520         allocate_and_create_tensor_map(d_a, M / BM, K / BK);

```

```

3521 CUtensorMap *tma_b =
3522     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
3523
3524 // Launch WGMMMA kernel
3525 // BLOCK_SWIZZLE=false → 2D grid, no swizzle
3526 size_t smem_bytes =
3527     K_STAGE * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
3528 cudaFuncSetAttribute(
3529     hgemma_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE,
3530     false>,
3531     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
3532
3533 dim3 block(NUM_THREADS);
3534 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
3535 hgemma_wgmma_stages_tn<64, 128, 16, BM, BN, BK, NUM_THREADS, K_STAGE, false>
3536     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
3537 check(cudaGetLastError(), "wgmma launch");
3538 check(cudaDeviceSynchronize(), "wgmma sync");
3539
3540 half *h_c = (half *)malloc(size_c);
3541 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
3542
3543 // Verify
3544 float max_err = 0.0f;
3545 for (int i = 0; i < M * N; i++) {
3546     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
3547     if (err > max_err)
3548         max_err = err;
3549 }
3550 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
3551     max_err < 1.0f ? "PASS" : "FAIL");
3552
3553 free(h_a);
3554 free(h_b);
3555 free(h_b_t);
3556 free(h_c);
3557 free(h_c_ref);
3558 cudaFree(d_a);
3559 cudaFree(d_b);
3560 cudaFree(d_b_t);
3561 cudaFree(d_c);
3562 cudaFree(tma_a);
3563 cudaFree(tma_b);
3564 cublasDestroy(handle);
3565 }
3566 #endif /* NOTES_V2_HAS_WGMMMA */
3567
3568
3569 static void test_flash_attn(int seqlen, int head_dim) {
3570     // FlashAttention-2 with split-Q, MMA m16n8k16
3571     int B = 1, H = 8;
3572
3573     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
3574
3575     srand(42);
3576     half *h_q = (half *)malloc(sz);
3577     half *h_k = (half *)malloc(sz);
3578     half *h_v = (half *)malloc(sz);
3579     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
3580         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3581         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3582         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
3583     }

```

```

3584 // CPU reference in FP32:  $O = \text{softmax}(Q @ K^T / \sqrt{d}) @ V$ 
3585 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
3586 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
3587 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
3588 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
3589 int count = B * H * seqlen * head_dim;
3590 for (int i = 0; i < count; i++) {
3591     ref_q[i] = __half2float(h_q[i]);
3592     ref_k[i] = __half2float(h_k[i]);
3593     ref_v[i] = __half2float(h_v[i]);
3594 }
3595
3596 float scale = 1.0f / sqrtf((float)head_dim);
3597 for (int bi = 0; bi < B * H; bi++) {
3598     for (int qi = 0; qi < seqlen; qi++) {
3599         //  $S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale$ 
3600         float smax = -INFINITY;
3601         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
3602         for (int kj = 0; kj < seqlen; kj++) {
3603             float s = 0.0f;
3604             for (int d = 0; d < head_dim; d++)
3605                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
3606                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
3607             S[kj] = s * scale;
3608             if (S[kj] > smax) smax = S[kj];
3609         }
3610         // softmax
3611         double sum_exp = 0.0;
3612         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
3613         float inv_sum = 1.0f / (float)sum_exp;
3614         //  $O[qi, :] = \sum_{kj} P[qi, kj] * V[kj, :]$ 
3615         for (int d = 0; d < head_dim; d++) {
3616             double o_acc = 0.0;
3617             for (int kj = 0; kj < seqlen; kj++)
3618                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
3619                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
3620             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
3621         }
3622         free(S);
3623     }
3624 }
3625 }
3626
3627 half *d_q, *d_k, *d_v, *d_o;
3628 check(cudaMalloc(&d_q, sz), "fa alloc Q");
3629 check(cudaMalloc(&d_k, sz), "fa alloc K");
3630 check(cudaMalloc(&d_v, sz), "fa alloc V");
3631 check(cudaMalloc(&d_o, sz), "fa alloc O");
3632 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
3633 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
3634 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
3635
3636 // Template params for kHeadDim=64, kStage=2
3637 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
3638 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
3639 constexpr int kMmaTileSeqLenQ = 4;
3640 constexpr int kMmaTileSeqLenK = 1;
3641 constexpr int kMmaTileSeqLenP = 4;
3642 constexpr int kMmaTileHeadDimV = 1;
3643 constexpr int kValTileSeqLenQ = 1;
3644 constexpr int kValTileSeqLenK = 8;
3645 constexpr int kValTileSeqLenP = 1;
3646 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);

```



```

3647
3648 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
3649 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
3650 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
3651                     kStageV * Bc * (kHeadDim + kPadV) +
3652                     Bc * (kHeadDim + kPadV)) * sizeof(half);
3653
3654 dim3 block(128);
3655 dim3 grid((seqlen + Br - 1) / Br, B * H);
3656
3657 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
3658 kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
3659 kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
3660 kStageV, kPadV>
3661 <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
3662 check(cudaGetLastError(), "fa launch");
3663 check(cudaDeviceSynchronize(), "fa sync");
3664
3665 half *h_o = (half *)malloc(sz);
3666 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
3667
3668 float max_err = 0.0f;
3669 for (int i = 0; i < count; i++) {
3670     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
3671     if (err > max_err) max_err = err;
3672 }
3673 printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
3674
3675 free(h_q); free(h_k); free(h_v); free(h_o);
3676 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
3677 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
3678 }
3679
3680
3681 int main(int argc, char *argv[]) {
3682 #if defined(NOTES_V2_HAS_WGMMA) || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) || defined(
    __CUDA_ARCH_FEAT_SM90_ALL)
3683     cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
3684 #endif
3685     int M = 1024, N = 1024, K = 1024;
3686     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
3687
3688     printf("=== notes-v2.cu verification harness ===\n");
3689     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
3690     printf("|-----|-----|-----|\n");
3691
3692     test_block_reduce(N);
3693     test_dot(N);
3694     test_relu(1024);
3695     test_elementwise(1024);
3696     test_histogram(1024);
3697     test_softmax(256); // softmax kernel requires N == blockDim.x
3698     test_rms_norm(8, 128);
3699     test_layer_norm(8, 128);
3700     test_rope(8, 128);
3701     test_mat_transpose(256, 256);
3702     test_mat_transpose_padded(256, 256);
3703     test_sgemv(256, 128);
3704     test_sgemm(M, N, K);
3705     test_hgemm_mma(M, N, K);
3706 #if defined(NOTES_V2_HAS_WGMMA) \
3707     || (defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 900) \
3708     || defined(__CUDA_ARCH_FEAT_SM90_ALL))

```

```

3709     test_hgemm_wgmma(M, N, K);
3710 #endif
3711     test_flash_attn(1024, 64);
3712
3713     printf("=== All tests done ===\n");
3714     return 0;
3715 }
3716
3717 // =====
3718 // Quick build & run reference
3719 // =====
3720 // # sm_89 单独编译 + 运行:
3721 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
3722 //
3723 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
3724 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a -DNOTES_V2_HAS_WGMMMA -lcublas -lcuda notes-v2.
    cu -o notes_v2_sm90.bin

```